

```
1 # ===== File: ghrun/src/bin/watchdog.rs =====
2 // watchdog.rs — ghrun watchdog process (~500 KB compiled)
3 // Responsibility: monitor ghrun.exe, respawn if dead.
4 // Pure std — zero external dependencies for minimal binary size.
5 //
6 // Usage: watchdog.exe <ghrun_path> [ghrun_args...]
7 // The ghrun_path should be absolute or relative to watchdog.exe location.
8
9 use std::{
10     env,
11     fs::{self, OpenOptions},
12     io::Write,
13     path::{Path, PathBuf},
14     process::{Child, Command},
15     thread,
16     time::{Duration, SystemTime, UNIX_EPOCH},
17 };
18
19 const HEARTBEAT: Duration = Duration::from_secs(5);
20 const DELAY: Duration = Duration::from_secs(1);
21 const MAX_RAPID: u32 = 10; // max rapid restarts before backing off
22 const BACKOFF: Duration = Duration::from_secs(30);
23
24 fn ts() -> u64 {
25     SystemTime::now().duration_since(UNIX_EPOCH).map(|d| d.as_secs()).unwrap_or(0)
26 }
27
28 fn log(lf: &Path, msg: &str) {
29     let line = format!("[watchdog {}] {}\\n", ts(), msg);
30     eprint!("{}", line);
31     if let Ok(mut f) = OpenOptions::new().create(true).append(true).open(lf) {
32         let _ = f.write_all(line.as_bytes());
33     }
34 }
35
36 fn log_path(ghrun: &Path) -> PathBuf {
37     let dir = ghrun.parent().unwrap_or(Path::new(".")).join("logs");
38     let _ = fs::create_dir_all(&dir);
39     dir.join("watchdog.log")
40 }
41
42 fn try_spawn(ghrun: &Path, args: &[String]) -> Option<Child> {
43     Command::new(ghrun).args(args).spawn().ok()
44 }
45
46 fn main() {
47     let argv: Vec<String> = env::args().collect();
48     if argv.len() < 2 {
49         eprintln!("Usage: watchdog <ghrun_path> [ghrun_args...]");
50         std::process::exit(1);
```

```
51     }
52     let ghrun = PathBuf::from(&argv[1]);
53     let args: Vec<String> = argv[2..].to_vec();
54     let lf = log_path(&ghrun);
55
56     log(&lf, &format!("start, target={}", ghrun.display()));
57
58     let mut child = try_spawn(&ghrun, &args);
59     let mut rapid = 0u32;
60
61     loop {
62         thread::sleep(HEARTBEAT);
63
64         let alive = match &mut child {
65             Some(c) => match c.try_wait() {
66                 Ok(None) => true,           // still running
67                 Ok(Some(st)) => {
68                     log(&lf, &format!("ghrun exited: {}", st));
69                     false
70                 }
71                 Err(e) => {
72                     log(&lf, &format!("try_wait err: {}", e));
73                     false
74                 }
75             },
76             None => false,
77         };
78
79         if !alive {
80             rapid += 1;
81             let wait = if rapid > MAX_RAPID { BACKOFF } else { DELAY };
82             log(&lf, &format!("respawn #{} in {}s", rapid, wait.as_secs()));
83             thread::sleep(wait);
84             child = try_spawn(&ghrun, &args);
85             if child.is_some() && rapid > MAX_RAPID {
86                 rapid = 0; // reset after successful backoff spawn
87             }
88         } else {
89             rapid = 0;
90         }
91     }
92 }
93
94 # ===== File: ghrun/src/ctx.rs =====
95 // ctx.rs — QDIR path context
96 // ghrun.exe lives at: <QDIR>/f/ghrun.exe
97 // Auto-infers QDIR from its own location, no env var required.
98 // But respects QDIR env var if explicitly set (e.g. during dev/CI).
99 //
100 // QDIR layout:
```

```

101 // QDIR/
102 // |—— qq.exe
103 // |—— f/ ← VS Code portable data (dataFolderName = "f")
104 // |—— ghrun.exe ← THIS BINARY
105 // |—— user-data/ ← VS Code user settings/keybindings
106 // |—— extensions/ ← pre-installed + user extensions
107 // |—— components/ ← python/ffmpeg/yt-dlp/git
108 // |—— goods/ ← gaea goods
109 // |—— tmp/ ← scratch space
110 // |—— cache/ ← download cache
111 // |—— logs/ ← runtime logs
112
113 use std::path::PathBuf;
114
115 #[derive(Debug, Clone)]
116 pub struct Ctx {
117     /// IDE portable root, e.g. /path/to/qqq-ide/
118     pub qdir: PathBuf,
119     /// f/ — VS Code portable data folder (triggers portable mode)
120     pub f: PathBuf,
121     /// f/user-data/ — VS Code user settings, keybindings, state
122     pub user_data: PathBuf,
123     /// f/extensions/ — pre-installed first-party + user extensions
124     pub extensions: PathBuf,
125     /// f/components/ — python / ffmpeg / yt-dlp / git
126     pub components: PathBuf,
127     /// f/goods/ — installed gaea goods
128     pub goods: PathBuf,
129     /// f/tmp/ — scratch space for downloads / extractions
130     pub tmp: PathBuf,
131     /// f/cache/ — download cache, ETag cache
132     pub cache: PathBuf,
133     /// f/logs/ — ghrun + component logs
134     pub logs: PathBuf,
135 }
136
137 impl Ctx {
138     pub fn detect() -> Self {
139         let qdir = Self::infer_qdir();
140
141         // Auto-detect layout:
142         // qq-shell-v2: userData/ + cache/ + logs/ at QDIR root
143         // q3 legacy: f/user-data/ + f/cache/ + f/logs/
144         let is_shell_v2 = qdir.join("userData").is_dir()
145             || qdir.join("cache").is_dir();
146
147         let (f, user_data, extensions, components, goods, tmp, cache, logs) =
148             if is_shell_v2 {
149                 // qq-shell-v2: flat layout, everything at QDIR level
150                 (

```

```
151         qdir.clone(),
152         qdir.join("userData"),
153         qdir.join("userData").join("extensions"),
154         qdir.join("userData").join("components"),
155         qdir.join("userData").join("goods"),
156         qdir.join("tmp"),
157         qdir.join("cache"),
158         qdir.join("logs"),
159     )
160 } else {
161     // q3 legacy: everything under f/ (VS Code portable data)
162     let f = qdir.join("f");
163     (
164         f.clone(),
165         f.join("user-data"),
166         f.join("extensions"),
167         f.join("components"),
168         f.join("goods"),
169         f.join("tmp"),
170         f.join("cache"),
171         f.join("logs"),
172     )
173 };
174
175 Ctx {
176     user_data,
177     extensions,
178     components,
179     goods,
180     tmp,
181     cache,
182     logs,
183     f,
184     qdir,
185 }
186 }
187
188 fn infer_qdir() -> PathBuf {
189     // Priority 1: explicit env var (dev / CI)
190     if let Ok(v) = std::env::var("QDIR") {
191         let p = PathBuf::from(&v);
192         if p.exists() { return p; }
193     }
194     // Priority 2: infer from ghrun.exe location
195     // ghrun.exe is expected at <QDIR>/f/ghrun[.exe]
196     if let Ok(exe) = std::env::current_exe() {
197         if let Some(f_dir) = exe.parent() { // → f/
198             if let Some(root) = f_dir.parent() { // → QDIR
199                 return root.to_path_buf();
200             }
201         }
202     }
```

```

201         }
202     }
203     // Priority 3: cwd fallback (local dev)
204     std::env::current_dir().unwrap_or_else(|_| PathBuf::from("."))
205 }
206
207 /// Ensure all QDIR directories exist.
208 pub fn init_dirs(&self) -> std::io::Result<> {
209     for d in [
210         &self.f, &self.user_data, &self.extensions,
211         &self.components, &self.goods, &self.tmp,
212         &self.cache, &self.logs,
213     ] {
214         std::fs::create_dir_all(d)?;
215     }
216     Ok(())
217 }
218
219 pub fn component_dir(&self, name: &str) -> PathBuf {
220     self.components.join(name)
221 }
222
223 pub fn goods_dir(&self, id: &str) -> PathBuf {
224     self.goods.join(id)
225 }
226 }
227
228 # ===== File: ghrun/src/ensure.rs =====
229 // ensure.rs — download, verify (SHA-256), extract component archives.
230 // All output is JSON lines to stdout so the Node.js extension can parse progress.
231
232 use crate::{ctx::Ctx, manifest::{ComponentManifest, PlatformEntry, current_platform, find_component}};
233 use sha2::{Digest, Sha256};
234 use std::{fs, io::{self, Read, Write}, path::{Path, PathBuf}};
235
236 // -----
237 // Progress events (JSON lines → stdout)
238 // -----
239
240 fn emit(obj: serde_json::Value) {
241     println!("{}", obj);
242     // flush — Node.js spawns ghrun with stdio piped
243     let _ = io::stdout().flush();
244 }
245
246 macro_rules! ev {
247     ($($k:expr => $v:expr),*) => {
248         serde_json::json!({ $($k: $v),* })
249     };
250 }

```

```

251
252 // -----
253 // Public entry point
254 // -----
255
256 /// `ghrun ensure <name>` — idempotent, safe to call multiple times.
257 pub fn ensure_component(ctx: &Ctx, name: &str) -> Result<(), String> {
258     let manifest = find_component(name, Some(&ctx.qdir))
259         .ok_or_else(|| format!("unknown component: {}", name))?;
260
261     let platform = current_platform();
262     let entry = manifest.platforms.get(platform)
263         .ok_or_else(|| format!("no entry for platform: {}", platform))?;
264
265     let dest = ctx.component_dir(name);
266
267     // Already installed? Check version marker.
268     let version_file = dest.join(".version");
269     if version_file.exists() {
270         let installed = fs::read_to_string(&version_file).unwrap_or_default();
271         if installed.trim() == manifest.version {
272             emit(ev!("event" => "already_installed", "component" => name, "version" => manifest.version))
273             return Ok(());
274         }
275     }
276
277     emit(ev!("event" => "start", "component" => name, "version" => manifest.version, "platform" => platf
278
279     // Create dest + tmp
280     fs::create_dir_all(&dest).map_err(|e| e.to_string())?;
281     let tmp_dir = ctx.tmp.join(format!("{}", name, &manifest.version));
282     fs::create_dir_all(&tmp_dir).map_err(|e| e.to_string())?;
283
284     let archive_path = download(ctx, entry, name)?;
285
286     if !entry.sha256.is_empty() {
287         verify_sha256(&archive_path, &entry.sha256)?;
288     } else {
289         emit(ev!("event" => "verify_skip", "reason" => "no sha256 in manifest"));
290     }
291
292     extract(entry, &archive_path, &dest, name)?;
293
294     // On unix: make bin executable
295     #[cfg(unix)]
296     if let Some(bin) = &entry.bin {
297         let bin_path = dest.join(bin);
298         if bin_path.exists() {
299             use std::os::unix::fs::PermissionsExt;
300             let _ = fs::set_permissions(&bin_path, fs::Permissions::from_mode(0o755));

```

```
301     }
302 }
303
304 // Write version marker
305 fs::write(&version_file, &manifest.version).map_err(|e| e.to_string())?;
306
307 // Announce final bin path
308 let bin_path = entry.bin.as_ref().map(|b| dest.join(b));
309 emit(ev!("event" => "done", "component" => name, "version" => manifest.version,
310     "path" => bin_path.as_ref().map(|p| p.to_string_lossy().to_string())));
311
312 // Clean up tmp archive
313 let _ = fs::remove_file(&archive_path);
314
315 Ok(())
316 }
317
318 /// Ensure multiple components in order.
319 pub fn ensure_all(ctx: &Ctx, names: &[&str]) -> Result<(), String> {
320     for name in names {
321         ensure_component(ctx, name)?;
322     }
323     Ok(())
324 }
325
326 // -----
327 // Download
328 // -----
329
330 fn download(ctx: &Ctx, entry: &PlatformEntry, name: &str) -> Result<PathBuf, String> {
331     let url = &entry.url;
332     let ext = if url.ends_with(".tar.gz") { ".tar.gz" }
333         else if url.ends_with(".zip") { ".zip" }
334         else { "" };
335     let archive_path = ctx.tmp.join(format!("{}", name, ext));
336
337     emit(ev!("event" => "download_start", "url" => url, "dest" => archive_path.to_string_lossy().to_stri
338
339     let client = reqwest::blocking::Client::builder()
340         .timeout(std::time::Duration::from_secs(600))
341         .build()
342         .map_err(|e| e.to_string())?;
343
344     let mut response = client.get(url)
345         .send()
346         .map_err(|e| format!("HTTP error: {}", e))?;
347
348     if !response.status().is_success() {
349         return Err(format!("download failed: HTTP {}", response.status()));
350     }
```

```
351
352     let total = response.content_length().unwrap_or(0);
353     let mut file = fs::File::create(&archive_path).map_err(|e| e.to_string())?;
354     let mut downloaded: u64 = 0;
355     let mut buf = vec![0u8; 65536];
356
357     loop {
358         let n = response.read(&mut buf).map_err(|e| e.to_string())?;
359         if n == 0 { break; }
360         file.write_all(&buf[..n]).map_err(|e| e.to_string())?;
361         downloaded += n as u64;
362         if total > 0 {
363             let pct = (downloaded * 100 / total) as u8;
364             // emit every 5%
365             if pct % 5 == 0 {
366                 emit(ev!("event" => "progress", "downloaded" => downloaded, "total" => total, "percent"
367                     })
368             }
369         }
370
371         emit(ev!("event" => "download_done", "bytes" => downloaded));
372         Ok(archive_path)
373     }
374
375 // -----
376 // SHA-256 verify
377 // -----
378
379 fn verify_sha256(path: &Path, expected: &str) -> Result<(), String> {
380     let mut file = fs::File::open(path).map_err(|e| e.to_string())?;
381     let mut hasher = Sha256::new();
382     let mut buf = vec![0u8; 65536];
383     loop {
384         let n = file.read(&mut buf).map_err(|e| e.to_string())?;
385         if n == 0 { break; }
386         hasher.update(&buf[..n]);
387     }
388     let got = hex::encode(hasher.finalize());
389     if got.to_lowercase() != expected.to_lowercase() {
390         return Err(format!("SHA-256 mismatch: got {} expected {}", got, expected));
391     }
392     emit(ev!("event" => "verify_ok", "sha256" => got));
393     Ok(())
394 }
395
396 // -----
397 // Extract
398 // -----
399
400 fn extract(entry: &PlatformEntry, archive: &Path, dest: &Path, name: &str) -> Result<(), String> {
```



```

401     match entry.kind.as_str() {
402         "binary" => {
403             // Raw binary — copy to dest/<bin>
404             let bin_name = entry.bin.as_deref().unwrap_or(name);
405             let out = dest.join(bin_name);
406             fs::copy(archive, &out).map_err(|e| e.to_string())?;
407             emit(ev!("event" => "extract_done", "path" => out.to_string_lossy().to_string()));
408         }
409         "zip" => {
410             let file = fs::File::open(archive).map_err(|e| e.to_string())?;
411             let mut zip = zip::ZipArchive::new(file).map_err(|e| e.to_string())?;
412             let strip = detect_common_prefix_zip(&mut zip);
413             for i in 0..zip.len() {
414                 let mut zf = zip.by_index(i).map_err(|e| e.to_string())?;
415                 let raw_name = zf.name().to_string();
416                 let rel = strip_prefix(&raw_name, &strip);
417                 if rel.is_empty() || rel.ends_with('/') { continue; }
418                 let out = dest.join(rel);
419                 if let Some(p) = out.parent() { fs::create_dir_all(p).map_err(|e| e.to_string())?; }
420                 let mut outf = fs::File::create(&out).map_err(|e| e.to_string())?;
421                 io::copy(&mut zf, &mut outf).map_err(|e| e.to_string())?;
422             }
423             emit(ev!("event" => "extract_done", "format" => "zip", "dest" => dest.to_string_lossy().to_s
424         }
425         "tar.gz" => {
426             let file = fs::File::open(archive).map_err(|e| e.to_string())?;
427             let gz = flate2::read::GzDecoder::new(file);
428             let mut tar = tar::Archive::new(gz);
429             tar.set_preserve_permissions(true);
430             // Determine strip prefix from first entry
431             let file2 = fs::File::open(archive).map_err(|e| e.to_string())?;
432             let gz2 = flate2::read::GzDecoder::new(file2);
433             let mut tar2 = tar::Archive::new(gz2);
434             let strip = detect_common_prefix_tar(&mut tar2);
435             // Re-open for actual extraction
436             let file3 = fs::File::open(archive).map_err(|e| e.to_string())?;
437             let gz3 = flate2::read::GzDecoder::new(file3);
438             let mut tar3 = tar::Archive::new(gz3);
439             for entry_res in tar3.entries().map_err(|e| e.to_string())? {
440                 let mut entry = entry_res.map_err(|e| e.to_string())?;
441                 let raw_path = entry.path().map_err(|e| e.to_string())?.to_string_lossy().to_string();
442                 let rel = strip_prefix(&raw_path, &strip);
443                 if rel.is_empty() || rel.ends_with('/') { continue; }
444                 let out = dest.join(&rel);
445                 if let Some(p) = out.parent() { fs::create_dir_all(p).map_err(|e| e.to_string())?; }
446                 entry.unpack(&out).map_err(|e| e.to_string())?;
447             }
448             emit(ev!("event" => "extract_done", "format" => "tar.gz", "dest" => dest.to_string_lossy().t
449         }
450     other => return Err(format!("unknown archive kind: {}", other)),

```

```

451     }
452     Ok(())
453 }
454
455 // -----
456 // Helpers: detect common archive prefix to strip
457 // -----
458
459 fn detect_common_prefix_zip(zip: &mut zip::ZipArchive<fs::File>) -> String {
460     if zip.len() == 0 { return String::new(); }
461     let first = zip.by_index(0).ok().map(|f| f.name().to_string()).unwrap_or_default();
462     if let Some(idx) = first.find('/') { first[..=idx].to_string() } else { String::new() }
463 }
464
465 fn detect_common_prefix_tar<R: Read>(tar: &mut tar::Archive<R>) -> String {
466     for entry in tar.entries().into_iter().flatten().flatten() {
467         if let Ok(p) = entry.path() {
468             let s = p.to_string_lossy().to_string();
469             if let Some(idx) = s.find('/') { return s[..=idx].to_string(); }
470         }
471         break;
472     }
473     String::new()
474 }
475
476 fn strip_prefix<'a>(path: &'a str, prefix: &str) -> &'a str {
477     if !prefix.is_empty() && path.starts_with(prefix) {
478         &path[prefix.len()..]
479     } else {
480         path
481     }
482 }
483
484 # ===== File: ghrun/src/install.rs =====
485 // install.rs — gaea goods installation.
486 // A "good" is anything in the gaea marketplace: tools, games, language packs, etc.
487 // Each good has a gaea.json (our protocol) and lives in f/goods/<id>/.
488
489 use crate::ctx::Ctx;
490 use serde::{Deserialize, Serialize};
491 use std::{fs, path::PathBuf};
492
493 fn emit(obj: serde_json::Value) {
494     println!("{}", obj);
495     let _ = std::io::Write::flush(&mut std::io::stdout());
496 }
497
498 // -----
499 // gaea.json subset we care about at install time
500 // -----

```

```

501
502 #[derive(Debug, Serialize, Deserialize)]
503 pub struct GaeaManifest {
504     pub id:          String,
505     pub name:         String,
506     pub version:      String,
507     pub install:      InstallInfo,
508 }
509
510 #[derive(Debug, Serialize, Deserialize)]
511 pub struct InstallInfo {
512     pub download_url: String,
513     pub sha256:        Option<String>,
514     pub size_mb:        Option<f64>,
515     pub target_dir:     Option<String>,    // QDIR_BUILTIN | QDIR_GOODS
516 }
517
518 // -----
519 // Install a good from its gaea.json URL or local path
520 // -----
521
522 /// `ghrun install <download_url>` — fetch gaea.json, then download + extract good.
523 /// Idempotent: checks version file before re-downloading.
524 pub fn install_good_from_url(ctx: &Ctx, manifest_url: &str) -> Result<(), String> {
525     emit(serde_json::json!({"event": "install_start", "url": manifest_url}));
526
527     let client = reqwest::blocking::Client::builder()
528         .timeout(std::time::Duration::from_secs(30))
529         .build()
530         .map_err(|e| e.to_string())?;
531
532     let body = client.get(manifest_url).send().map_err(|e| e.to_string())?
533         .text().map_err(|e| e.to_string())?;
534
535     let manifest: GaeaManifest = serde_json::from_str(&body)
536         .map_err(|e| format!("invalid gaea.json: {}", e))?;
537
538     install_good(ctx, &manifest)
539 }
540
541 /// Install a good from a parsed GaeaManifest.
542 pub fn install_good(ctx: &Ctx, manifest: &GaeaManifest) -> Result<(), String> {
543     let target = match manifest.install.target_dir.as_deref() {
544         Some("QDIR_BUILTIN") => ctx.extensions.join(&manifest.id),
545         _                     => ctx.goods.join(&manifest.id),
546     };
547
548     // Already installed at same version?
549     let ver_file = target.join(".version");
550     if ver_file.exists() {

```

```

551         let installed = fs::read_to_string(&ver_file).unwrap_or_default();
552         if installed.trim() == manifest.version {
553             emit(serde_json::json!({"event": "already_installed", "id": manifest.id, "version": manifest.version}));
554             return Ok(());
555         }
556     }
557
558     emit(serde_json::json!({"event": "install_download", "id": manifest.id, "version": manifest.version}));
559
560     let tmp_archive = ctx.tmp.join(format!("{}-{}.zip", manifest.id, manifest.version));
561     download_file(&manifest.install.download_url, &tmp_archive)?;
562
563     if let Some(sha) = &manifest.install.sha256 {
564         if !sha.is_empty() {
565             verify_sha256(&tmp_archive, sha)?;
566         }
567     }
568
569     fs::create_dir_all(&target).map_err(|e| e.to_string())?;
570     extract_zip(&tmp_archive, &target)?;
571
572     fs::write(&ver_file, &manifest.version).map_err(|e| e.to_string())?;
573     let _ = fs::remove_file(&tmp_archive);
574
575     emit(serde_json::json!({"event": "install_done", "id": manifest.id, "version": manifest.version,
576                             "path": target.to_string_lossy().to_string()}));
577     Ok(())
578 }
579
580 // -----
581 // List installed goods
582 // -----
583
584 pub fn list_goods(ctx: &Ctx) {
585     let dirs = [
586         ("builtin", &ctx.extensions),
587         ("goods", &ctx.goods),
588     ];
589     for (label, dir) in &dirs {
590         if let Ok(entries) = fs::read_dir(dir) {
591             for entry in entries.flatten() {
592                 let path = entry.path();
593                 let version = fs::read_to_string(path.join(".version")).unwrap_or_else(|_| "?".into());
594                 let id = path.file_name().unwrap_or_default().to_string_lossy().to_string();
595                 emit(serde_json::json!({"event": "list", "tier": label, "id": id, "version": version.trim()}));
596             }
597         }
598     }
599 }
600

```

```

601 // -----
602 // Helpers (reused from ensure.rs logic — kept local to avoid coupling)
603 // -----
604
605 fn download_file(url: &str, dest: &PathBuf) -> Result<(), String> {
606     use std::io::{Read, Write};
607     let client = reqwest::blocking::Client::builder()
608         .timeout(std::time::Duration::from_secs(600))
609         .build().map_err(|e| e.to_string())?;
610     let mut resp = client.get(url).send().map_err(|e| e.to_string())?;
611     if !resp.status().is_success() {
612         return Err(format!("HTTP {}", resp.status()));
613     }
614     let mut f = fs::File::create(dest).map_err(|e| e.to_string())?;
615     let mut buf = vec![0u8; 65536];
616     let total = resp.content_length().unwrap_or(0);
617     let mut got: u64 = 0;
618     loop {
619         let n = resp.read(&mut buf).map_err(|e| e.to_string())?;
620         if n == 0 { break; }
621         f.write_all(&buf[..n]).map_err(|e| e.to_string())?;
622         got += n as u64;
623         if total > 0 && got % (1 << 20) == 0 {
624             emit(serde_json::json!({"event": "progress", "downloaded": got, "total": total}));
625         }
626     }
627     Ok(())
628 }
629
630 fn verify_sha256(path: &PathBuf, expected: &str) -> Result<(), String> {
631     use sha2::Digest, Sha256;
632     use std::io::Read;
633     let mut f = fs::File::open(path).map_err(|e| e.to_string())?;
634     let mut h = Sha256::new();
635     let mut buf = vec![0u8; 65536];
636     loop {
637         let n = f.read(&mut buf).map_err(|e| e.to_string())?;
638         if n == 0 { break; }
639         h.update(&buf[..n]);
640     }
641     let got = hex::encode(h.finalize());
642     if got.to_lowercase() != expected.to_lowercase() {
643         return Err(format!("sha256 mismatch: {} vs {}", got, expected));
644     }
645     Ok(())
646 }
647
648 fn extract_zip(archive: &PathBuf, dest: &PathBuf) -> Result<(), String> {
649     use std::io;
650     let f = fs::File::open(archive).map_err(|e| e.to_string())?;

```

```

651     let mut z = zip::ZipArchive::new(f).map_err(|e| e.to_string())?;
652     // detect and strip top-level directory prefix
653     let prefix = if z.len() > 0 {
654         z.by_index(0).ok()
655         .and_then(|e| { let n = e.name().to_string(); n.find('/').map(|i| n[..i].to_string()) })
656         .unwrap_or_default()
657     } else { String::new() };
658     for i in 0..z.len() {
659         let mut zf = z.by_index(i).map_err(|e| e.to_string())?;
660         let raw = zf.name().to_string();
661         let rel = if !prefix.is_empty() && raw.starts_with(&prefix) { &raw[prefix.len()..] } else { &raw
662         if rel.is_empty() || rel.ends_with('/') { continue; }
663         let out = dest.join(rel);
664         if let Some(p) = out.parent() { fs::create_dir_all(p).map_err(|e| e.to_string())?; }
665         let mut outf = fs::File::create(&out).map_err(|e| e.to_string())?;
666         io::copy(&mut zf, &mut outf).map_err(|e| e.to_string())?;
667     }
668     Ok(())
669 }
670
671 # ===== File: ghrun/src/lsp_daemon.rs =====
672 // lsp_daemon.rs — LSP TCP broker for multi-Shell sharing
673 //
674 // Spawns LSP servers and exposes them via TCP. One process per language
675 // shared by all IDE instances. Pure byte forwarding with Content-Length
676 // header framing — no JSON parsing, no protocol awareness.
677 //
678 // Architecture:
679 //   Shell TCP clients — TCP —> ghrun daemon <— stdin/stdout —> LSP server
680 //   (Content-Length frames)   (byte forwarding)           (gopls/pyright/...)
681
682 use std::io::{BufRead, BufReader, Read, Write};
683 use std::net::{TcpListener, TcpStream};
684 use std::process::{Child, Command, Stdio};
685 use std::sync::{Arc, Mutex};
686 use std::thread;
687 use std::time::Duration;
688
689 #[cfg(windows)]
690 use std::os::windows::io::AsRawHandle;
691
692 #[cfg(windows)]
693 extern "system" {
694     fn SetPriorityClass(handle: isize, priority: u32) -> i32;
695 }
696
697 #[cfg(windows)]
698 const BELOW_NORMAL_PRIORITY_CLASS: u32 = 0x00004000;
699
700 use crate::ctx::Ctx;

```

```
701 use crate::manifest;
702
703 /// Well-known port mapping: component → TCP port.
704 /// Shell uses the same mapping to connect.
705 pub fn port_for(name: &str) -> u16 {
706     match name {
707         "lsp/gopls"      => 9801,
708         "lsp/pyright"    => 9802,
709         "lsp/clangd"     => 9803,
710         "lsp/rust-analyzer" => 9804,
711         _ => 0,
712     }
713 }
714
715 /// Start all installed LSP servers with TCP listeners.
716 /// Blocks until process killed (Ctrl+C / SIGTERM).
717 pub fn run(ctx: &Ctx) -> Result<(), String> {
718     let mut handles: Vec<LspHandle> = Vec::new();
719
720     let all_components = manifest::all_components(Some(&ctx.qdir));
721     for comp in &all_components {
722         if !comp.name.starts_with("lsp/") {
723             continue;
724         }
725         let port = port_for(&comp.name);
726         if port == 0 {
727             continue;
728         }
729
730         let (dir_name, bin_name) = match manifest::find_bin(&comp.name, Some(&ctx.qdir)) {
731             Some(v) => v,
732             None => {
733                 eprintln!("[lsp-daemon] skip {}: no manifest entry", comp.name);
734                 continue;
735             }
736         };
737         let bin_path = ctx.component_dir(&dir_name).join(&bin_name);
738         if !bin_path.exists() {
739             eprintln!(
740                 "[lsp-daemon] skip {}: not installed at {}",
741                 comp.name,
742                 bin_path.display()
743             );
744             continue;
745         }
746
747         let args: Vec<&str> = match comp.name.as_str() {
748             "lsp/pyright" => vec!["--stdio"],
749             _ => vec![],
750         };
751     }
```

```

751
752     eprintln!(
753         "[lsp-daemon] starting {} : {} ({})",
754         comp.name,
755         port,
756         bin_path.display()
757     );
758
759     match launch_one(&bin_path, &args, port) {
760         Ok(h) => handles.push(h),
761         Err(e) => eprintln!("[lsp-daemon] {}: {}", comp.name, e),
762     }
763 }
764
765 if handles.is_empty() {
766     return Err("no LSP components installed. Run: ghrun ensure lsp/gopls lsp/pyright lsp/clangd lsp/");
767 }
768
769 // Signal readiness to stdout (Shell reads this to confirm startup)
770 let ready_ports: Vec<u16> = handles.iter().map(|h| h.port).collect();
771 println!(
772     "{}",
773     serde_json::json!({
774         "event": "lsp_daemon_ready",
775         "ports": ready_ports,
776     })
777 );
778
779 eprintln!(
780     "[lsp-daemon] {} server(s) running, waiting for connections...",
781     handles.len()
782 );
783
784 // Block forever — all work happens in background threads.
785 loop {
786     thread::sleep(Duration::from_secs(3600));
787 }
788 }
789
790 /// One launched LSP → TCP broker.
791 struct LspHandle {
792     port: u16,
793     _child: Arc<Mutex<Option<Child>>>,
794 }
795
796 /// Spawn LSP process + TCP listener + forwarding threads.
797 fn launch_one(
798     bin_path: &std::path::Path,
799     extra_args: &[&str],
800     port: u16,

```



```

801 ) -> Result<LspHandle, String> {
802     let mut child = Command::new(bin_path)
803         .args(extra_args)
804         .stdin(Stdio::piped())
805         .stdout(Stdio::piped())
806         .stderr(Stdio::inherit())
807         .spawn()
808         .map_err(|e| format!("spawn {:?}: {}", bin_path, e))?;
809
810     // Set below-normal priority so LSP never starves the IDE
811     #[cfg(windows)]
812     {
813         unsafe {
814             SetPriorityClass(child.as_raw_handle() as isize, BELOW_NORMAL_PRIORITY_CLASS);
815         }
816     }
817
818     let child_stdin = child.stdin.take().unwrap();
819     let child_stdout = child.stdout.take().unwrap();
820     let child_holder = Arc::new(Mutex::new(Some(child)));
821
822     let listener = TcpListener::bind(("127.0.0.1", port))
823         .map_err(|e| format!("bind {:?}: {}", port, e))?;
824     // Default blocking mode — accept() blocks until connection, zero-CPU polling
825
826     // Shared mutable client pool
827     let clients: Arc<Mutex<Vec<TcpStream>>> = Arc::new(Mutex::new(Vec::new()));
828
829     // — Thread A: LSP stdout → broadcast to all TCP clients —
830     let clients_a = clients.clone();
831     let child_a = child_holder.clone();
832     thread::spawn(move || {
833         pump_stdout_to_clients(BufReader::new(child_stdout), clients_a, child_a, port);
834     });
835
836     // — Thread B: accept TCP connections + spawn per-client readers —
837     let clients_b = clients.clone();
838     let stdin_arc = Arc::new(Mutex::new(child_stdin));
839     let child_b = child_holder.clone();
840     thread::spawn(move || {
841         accept_loop(listener, clients_b, stdin_arc, child_b, port);
842     });
843
844     Ok(LspHandle {
845         port,
846         _child: child_holder,
847     })
848 }
849
850 // -----

```

```

851 // Thread A: LSP stdout → all TCP clients
852 // -----
853
854 fn pump_stdout_to_clients(
855     mut reader: BufReader<impl Read + Send + 'static>,
856     clients: Arc<Mutex<Vec<TcpStream>>>,
857     _child: Arc<Mutex<Option<Child>>>,
858     port: u16,
859 ) {
860     loop {
861         let frame = match read_lsp_frame(&mut reader) {
862             Some(f) => f,
863             None => {
864                 eprintln!("[lsp-daemon :{}] LSP stdout closed", port);
865                 return;
866             }
867         };
868
869         // Broadcast to all connected clients — remove dead ones
870         let mut dead: Vec<usize> = Vec::new();
871         {
872             let mut guard = clients.lock().unwrap();
873             for (i, client) in guard.iter_mut().enumerate() {
874                 if client.write_all(&frame).is_err() {
875                     dead.push(i);
876                 }
877             }
878             for i in dead.into_iter().rev() {
879                 guard.remove(i);
880             }
881         }
882     }
883 }
884
885 // -----
886 // Thread B: accept TCP clients → spawn per-client forwarders
887 // -----
888
889 fn accept_loop(
890     listener: TcpListener,
891     clients: Arc<Mutex<Vec<TcpStream>>>,
892     stdin: Arc<Mutex<std::process::ChildStdin>>,
893     _child: Arc<Mutex<Option<Child>>>,
894     port: u16,
895 ) {
896     // Blocking accept — zero CPU, zero latency (no polling)
897     loop {
898         match listener.accept() {
899             Ok((stream, addr)) => {
900                 eprintln!("[lsp-daemon :{}] client connected from {}", port, addr);

```

```

901         {
902             let mut guard = clients.lock().unwrap();
903             guard.push(stream.try_clone().unwrap());
904         }
905         // Per-client: read TCP → write LSP stdin
906         let s = stdin.clone();
907         thread::spawn(move || {
908             pump_client_to_stdin(BufReader::new(stream), s, port);
909         });
910     }
911     Err(e) => {
912         eprintln!("[lsp-daemon :{}] accept error: {}", port, e);
913         thread::sleep(Duration::from_secs(1));
914     }
915 }
916 }
917 }
918
919 /// Read Content-Length framed messages from one TCP client → LSP stdin.
920 fn pump_client_to_stdin(
921     mut reader: BufReader<TcpStream>,
922     stdin: Arc<Mutex<std::process::ChildStdin>>,
923     port: u16,
924 ) {
925     loop {
926         let frame = match read_lsp_frame(&mut reader) {
927             Some(f) => f,
928             None => {
929                 eprintln!("[lsp-daemon :{}] client disconnected", port);
930                 return;
931             }
932         };
933         let mut guard = stdin.lock().unwrap();
934         if guard.write_all(&frame).is_err() {
935             eprintln!("[lsp-daemon :{}] write to LSP stdin failed", port);
936             return;
937         }
938         let _ = guard.flush();
939     }
940 }
941
942 // -----
943 // Content-Length header aware frame reader
944 // -----
945
946 /// Read one complete LSP frame (Content-Length header + body) from a BufRead.
947 /// Returns None on EOF or fatal error.
948 fn read_lsp_frame<R: BufRead>(reader: &mut R) -> Option<Vec<u8>> {
949     // 1. Read headers until \r\n\r\n
950     let mut header_bytes = Vec::new();

```

```

951     loop {
952         let mut line = Vec::new();
953         match reader.read_until(b'\n', &mut line) {
954             Ok(0) => return None, // EOF
955             Ok(_) => {
956                 let is_blank = line == b"\r\n" || line == b"\n";
957                 header_bytes.extend_from_slice(&line);
958                 if is_blank {
959                     break;
960                 }
961             }
962             Err(_) => return None,
963         }
964     }
965
966     // 2. Parse Content-Length
967     let header_str = String::from_utf8_lossy(&header_bytes);
968     let content_length = header_str
969         .lines()
970         .find(|l| l.to_lowercase().starts_with("content-length:"))
971         .and_then(|l| l.split(':').nth(1))
972         .and_then(|v| v.trim().parse::<usize>().ok())?;
973
974     // Reject oversized frames (prevents OOM from corrupted/malicious headers)
975     if content_length > 50 * 1024 * 1024 {
976         return None;
977     }
978
979     // 3. Read body
980     let mut body = vec![0u8; content_length];
981     if reader.read_exact(&mut body).is_err() {
982         return None;
983     }
984
985     // 4. Return complete frame (header + body)
986     header_bytes.extend_from_slice(&body);
987     Some(header_bytes)
988 }
989
990 # ===== File: ghrun/src/main.rs =====
991 // main.rs — ghrun: qqide component manager & gaea goods installer
992 //
993 // Protocol: all output is JSON lines to stdout.
994 //   {"event": "start", "component": "ffmpeg", "version": "7.1"}
995 //   {"event": "progress", "downloaded": 1048576, "total": 52428800, "percent": 2}
996 //   {"event": "done", "component": "ffmpeg", "path": "/path/to/ffmpeg.exe"}
997 //   {"event": "error", "msg": "..."}
998 //
999 // The Node.js extension spawns ghrun and parses stdout line-by-line.
1000 // stderr is for human-readable debug info only.

```

```
1001
1002 use clap::{Parser, Subcommand};
1003 use std::path::PathBuf;
1004 use std::fs;
1005 use std::io::Read;
1006
1007 mod ctx;
1008 #[macro_use]
1009 mod ensure;
1010 mod install;
1011 mod lsp_daemon;
1012 mod manifest;
1013 mod spawn;
1014
1015 use ctx::Ctx;
1016
1017 // -----
1018 // CLI definition
1019 // -----
1020
1021 #[derive(Parser)]
1022 #[command(name = "ghrun", version, about = "qqide component manager")]
1023 struct Cli {
1024     #[command(subcommand)]
1025     cmd: Cmd,
1026 }
1027
1028 #[derive(Subcommand)]
1029 enum Cmd {
1030     /// Initialize QDIR directory structure
1031     Init,
1032
1033     /// Ensure a component is installed (idempotent)
1034     /// Examples: ghrun ensure ffmpeg | ghrun ensure python yt-dlp
1035     Ensure {
1036         /// Component name(s): ffmpeg, python, yt-dlp, git
1037         #[arg(required = true)]
1038         components: Vec<String>,
1039     },
1040
1041     /// Install a gaea good from its gaea.json URL
1042     Install {
1043         /// URL to gaea.json manifest
1044         url: String,
1045     },
1046
1047     /// Upgrade all installed components to latest versions
1048     Upgrade,
1049
1050     /// Upgrade shell-out/ from remote (download + verify + atomic swap + restart Electron)
```

```
1051     UpgradeShell {
1052         /// URL to shell-out.tar.xz
1053         url: String,
1054         /// Expected SHA-256 hex
1055         #[arg(long)]
1056         sha256: Option<String>,
1057         /// Path to the shell-out directory to replace
1058         #[arg(long)]
1059         target: String,
1060         /// Path to electron.exe to restart
1061         #[arg(long)]
1062         electron: String,
1063     },
1064
1065     /// List installed components and goods
1066     List,
1067
1068     /// Print QDIR path info (useful for debugging)
1069     Info,
1070
1071     /// Start the LSP TCP daemon (byte-forwarding broker for multi-Shell sharing)
1072     LspDaemon,
1073
1074     /// Resolve a component's binary path (used by qz-spawn / LSP bridge)
1075     /// Also includes "port" when the component is an LSP server
1076     Which {
1077         /// Component name: ffmpeg, python, lsp/gopls, lsp/clangd, etc.
1078         name: String,
1079     },
1080
1081     /// Spawn a one-shot subprocess with deadline watchdog (stdin JSON RPC)
1082     /// Protocol matches qz-spawn.ts ghrunTier + engines/runner.py
1083     Spawn {
1084         /// Always pass this flag (reserved, protocol compat)
1085         #[arg(long)]
1086         json: bool,
1087     },
1088
1089     /// Print QDIR root path (for debugging / env detection)
1090     Root,
1091
1092     /// Health check — verify ghrun is functional (for CI / startup probe)
1093     Doctor,
1094 }
1095
1096 // -----
1097 // Main
1098 // -----
1099
1100 fn main() {
```

```

1101     let cli = Cli::parse();
1102     let ctx = Ctx::detect();
1103
1104     let result = match cli.cmd {
1105         Cmd::Init => cmd_init(&ctx),
1106         Cmd::Ensure { components } => cmd_ensure(&ctx, &components),
1107         Cmd::Install { url } => cmd_install(&ctx, &url),
1108         Cmd::Upgrade => cmd_upgrade(&ctx),
1109         Cmd::UpgradeShell { url, sha256, target, electron } => cmd_upgrade_shell(&ctx, &url, &sha256, &t
1110         Cmd::List => { install::list_goods(&ctx); Ok(()) },
1111         Cmd::Info => cmd_info(&ctx),
1112         Cmd::LspDaemon => lsp_daemon::run(&ctx),
1113         Cmd::Which { name } => cmd_which(&ctx, &name),
1114         Cmd::Spawn { .. } => spawn::run(),
1115         Cmd::Root => cmd_root(&ctx),
1116         Cmd::Doctor => cmd_doctor(&ctx),
1117     };
1118
1119     if let Err(e) = result {
1120         let msg = serde_json::json!({"event":"error","msg": e});
1121         eprintln!("{}", msg);
1122         println!("{}", msg);
1123         std::process::exit(1);
1124     }
1125 }
1126
1127 // -----
1128 // Command implementations
1129 // -----
1130
1131 fn cmd_init(ctx: &Ctx) -> Result<(), String> {
1132     ctx.init_dirs().map_err(|e| e.to_string())?;
1133     let marker = ctx.f.join(".qq-portable");
1134     if !marker.exists() {
1135         std::fs::write(&marker, format!("ghrun init {}", chrono_now()))
1136         .map_err(|e| e.to_string())?;
1137     }
1138     println!("{}", serde_json::json!({"event":"init_done","qdir": ctx.qdir.to_string_lossy().to_string()
1139     Ok(())
1140 }
1141
1142 fn cmd_ensure(ctx: &Ctx, names: &[String]) -> Result<(), String> {
1143     ctx.init_dirs().map_err(|e| e.to_string())?;
1144     let refs: Vec<&str> = names.iter().map(|s| s.as_str()).collect();
1145     ensure::ensure_all(ctx, &refs)
1146 }
1147
1148 fn cmd_install(ctx: &Ctx, url: &str) -> Result<(), String> {
1149     ctx.init_dirs().map_err(|e| e.to_string())?;
1150     install::install_good_from_url(ctx, url)

```

```
1151 }
1152
1153 fn cmd_upgrade(ctx: &Ctx) -> Result<(), String> {
1154     let components = ["ffmpeg", "python", "yt-dlp", "git"];
1155     for name in &components {
1156         let ver_file = ctx.component_dir(name).join(".version");
1157         if ver_file.exists() {
1158             if let Err(e) = ensure::ensure_component(ctx, name) {
1159                 eprintln!("upgrade {}: {}", name, e);
1160             }
1161         }
1162     }
1163     println!("{}", serde_json::json!({"event": "upgrade_done"}));
1164     Ok(())
1165 }
1166
1167 fn cmd_upgrade_shell(ctx: &Ctx, url: &str, sha256: &Option<String>, target: &str, electron: &str) -> Res
1168     use std::process::Command;
1169
1170     let target_dir = PathBuf::from(target);
1171     if !target_dir.exists() {
1172         return Err(format!("target directory not found: {}", target));
1173     }
1174
1175     let staging = ctx.tmp.join("shell-upgrade");
1176     let _ = fs::remove_dir_all(&staging);
1177     fs::create_dir_all(&staging).map_err(|e| e.to_string())?;
1178
1179     let archive_path = staging.join("shell-out.tar.xz");
1180
1181     // 1) Download
1182     println!("{}", ev!("event" => "download_start", "url" => url));
1183     let resp = request::blocking::get(url)
1184         .map_err(|e| format!("download failed: {}", e))?;
1185     let total = resp.content_length();
1186     let bytes = resp.bytes().map_err(|e| format!("download read failed: {}", e))?;
1187     let downloaded = bytes.len() as u64;
1188     fs::write(&archive_path, &bytes).map_err(|e| e.to_string())?;
1189     println!("{}", ev!("event" => "download_done", "size" => downloaded));
1190
1191     // 2) Verify SHA-256
1192     if let Some(expected) = sha256 {
1193         if !expected.is_empty() {
1194             let actual = sha256_file(&archive_path)?;
1195             if !actual.eq_ignore_ascii_case(expected) {
1196                 return Err(format!("SHA-256 mismatch: expected {}, got {}", expected, actual));
1197             }
1198             println!("{}", ev!("event" => "verify_ok", "sha256" => actual));
1199         }
1200     }
```



```
1201
1202 // 3) Extract to staging/extracted
1203 let extract_dir = staging.join("extracted");
1204 fs::create_dir_all(&extract_dir).map_err(|e| e.to_string())?;
1205 extract_tar_xz(&archive_path, &extract_dir)?;
1206 println!("{}", ev!("event" => "extract_done"));
1207
1208 // 4) Signal ready — parent (Electron main process) should now quit
1209 println!("{}", ev!("event" => "ready_to_swap", "staging" => extract_dir.to_string_lossy().to_string(
1210
1211 // 5) Wait for parent process (Electron) to exit
1212 // On Windows, we watch the electron.exe process
1213 let electron_path = PathBuf::from(electron);
1214 if electron_path.exists() {
1215     let electron_name = electron_path.file_name()
1216         .map(|n| n.to_string_lossy().to_string())
1217         .unwrap_or_else(|| "electron.exe".to_string());
1218     println!("{}", ev!("event" => "waiting_for_exit", "process" => electron_name));
1219     // Poll until electron.exe is gone (max 30s)
1220     for _ in 0..60 {
1221         let running = is_process_running(&electron_name);
1222         if !running { break; }
1223         std::thread::sleep(std::time::Duration::from_millis(500));
1224     }
1225 } else {
1226     // Fallback: wait 2s for parent to die
1227     std::thread::sleep(std::time::Duration::from_secs(2));
1228 }
1229
1230 // 6) Atomic swap
1231 let backup = target_dir.with_extension("old");
1232 let _ = fs::remove_dir_all(&backup);
1233 fs::rename(&target_dir, &backup).map_err(|e| format!("backup failed: {}", e))?;
1234 // Copy extracted contents into target
1235 copy_dir_recursive(&extract_dir, &target_dir)?;
1236 let _ = fs::remove_dir_all(&backup);
1237 let _ = fs::remove_dir_all(&staging);
1238 println!("{}", ev!("event" => "swap_done"));
1239
1240 // 7) Restart Electron
1241 if electron_path.exists() {
1242     println!("{}", ev!("event" => "restarting"));
1243     Command::new(&electron_path)
1244         .arg(".")
1245         .spawn()
1246         .map_err(|e| format!("restart failed: {}", e))?;
1247 }
1248
1249 println!("{}", ev!("event" => "upgrade_shell_done"));
1250 Ok()
```

```
1251 }
1252
1253 fn sha256_file(path: &PathBuf) -> Result<String, String> {
1254     use sha2::Digest, Sha256;
1255     let mut file = fs::File::open(path).map_err(|e| e.to_string())?;
1256     let mut hasher = Sha256::new();
1257     let mut buf = [0u8; 8192];
1258     loop {
1259         let n = file.read(&mut buf).map_err(|e| e.to_string())?;
1260         if n == 0 { break; }
1261         hasher.update(&buf[..n]);
1262     }
1263     Ok(format!("{:x}", hasher.finalize()))
1264 }
1265
1266 fn extract_tar_xz(archive: &PathBuf, dest: &PathBuf) -> Result<(), String> {
1267     let file = fs::File::open(archive).map_err(|e| e.to_string())?;
1268     let xz = xz2::read::XzDecoder::new(file);
1269     let mut archive = tar::Archive::new(xz);
1270     archive.unpack(dest).map_err(|e| format!("extract failed: {}", e))?;
1271     Ok(())
1272 }
1273
1274 fn copy_dir_recursive(src: &PathBuf, dst: &PathBuf) -> Result<(), String> {
1275     fs::create_dir_all(dst).map_err(|e| e.to_string())?;
1276     for entry in fs::read_dir(src).map_err(|e| e.to_string())? {
1277         let entry = entry.map_err(|e| e.to_string())?;
1278         let ty = entry.file_type().map_err(|e| e.to_string())?;
1279         let name = entry.file_name();
1280         let dst_path = dst.join(&name);
1281         if ty.is_dir() {
1282             copy_dir_recursive(&entry.path(), &dst_path)?;
1283         } else {
1284             fs::copy(entry.path(), &dst_path).map_err(|e| e.to_string())?;
1285         }
1286     }
1287     Ok(())
1288 }
1289
1290 #[cfg(windows)]
1291 fn is_process_running(name: &str) -> bool {
1292     use std::process::Command;
1293     let output = Command::new("tasklist")
1294         .args(["/FI", &format!("IMAGENAME eq {}", name)])
1295         .output();
1296     match output {
1297         Ok(o) => {
1298             let stdout = String::from_utf8_lossy(&o.stdout);
1299             stdout.contains(name)
1300         }
1301     }
```

```
1301     Err(_) => false,
1302   }
1303 }
1304
1305 #[cfg(not(windows))]
1306 fn is_process_running(name: &str) -> bool {
1307     use std::process::Command;
1308     let output = Command::new("pgrep")
1309         .arg(name)
1310         .output();
1311     match output {
1312         Ok(o) => o.status.success(),
1313         Err(_) => false,
1314     }
1315 }
1316
1317 fn cmd_info(ctx: &Ctx) -> Result<(), String> {
1318     println!("{}", serde_json::json!({
1319         "event": "info",
1320         "qdir": ctx.qdir.to_string_lossy().to_string(),
1321         "f": ctx.f.to_string_lossy().to_string(),
1322         "components": ctx.components.to_string_lossy().to_string(),
1323         "goods": ctx.goods.to_string_lossy().to_string(),
1324         "builtin": ctx.extensions.to_string_lossy().to_string(),
1325         "platform": manifest::current_platform(),
1326     }));
1327     Ok(())
1328 }
1329
1330 fn cmd_which(ctx: &Ctx, name: &str) -> Result<(), String> {
1331     let (dir_name, bin_name) = manifest::find_bin(name, Some(&ctx.qdir))
1332         .ok_or_else(|| format!("component '{}' not found in manifest", name))?;
1333     let dir = ctx.component_dir(&dir_name);
1334     let full_path = dir.join(&bin_name);
1335     let path_str = full_path.to_string_lossy().to_string();
1336
1337     let port = lsp_daemon::port_for(name);
1338
1339     println!("{}", serde_json::json!({
1340         "event": "which",
1341         "name": name,
1342         "path": path_str,
1343         "dir": dir.to_string_lossy().to_string(),
1344         "bin": bin_name,
1345         "port": if port > 0 { serde_json::json!(port) } else { serde_json::Value::Null },
1346     }));
1347     Ok(())
1348 }
1349
1350 fn chrono_now() -> String {
```

```

1351     use std::time::{SystemTime, UNIX_EPOCH};
1352     let secs = SystemTime::now().duration_since(UNIX_EPOCH)
1353         .map(|d| d.as_secs()).unwrap_or(0);
1354     format!("unix={}", secs)
1355 }
1356
1357 fn cmd_root(ctx: &Ctx) -> Result<(), String> {
1358     let exe = std::env::current_exe().unwrap_or_default();
1359     println!("{}", serde_json::json!({
1360         "event": "root",
1361         "qdir": ctx.qdir.to_string_lossy(),
1362         "ghrun": exe.to_string_lossy(),
1363     }));
1364     Ok(())
1365 }
1366
1367 fn cmd_doctor(ctx: &Ctx) -> Result<(), String> {
1368     let exe = std::env::current_exe().unwrap_or_default();
1369     println!("{}", serde_json::json!({
1370         "event": "doctor",
1371         "status": "ok",
1372         "qdir": ctx.qdir.to_string_lossy(),
1373         "ghrun": exe.to_string_lossy(),
1374         "platform": std::env::consts::OS,
1375         "arch": std::env::consts::ARCH,
1376         "version": env!("CARGO_PKG_VERSION"),
1377     }));
1378     Ok(())
1379 }
1380
1381 # ===== File: ghrun/src/manifest.rs =====
1382 // manifest.rs — component and goods manifest definitions.
1383 //
1384 // Resolution order (first wins):
1385 // 1. <QDIR>/userData/manifest.json (user override, survives upgrades)
1386 // 2. <QDIR>/engines/manifest.json (shipped with ghrun, updated by CI)
1387 // 3. Hardcoded builtin fallback (compiled into binary, always available)
1388 //
1389 // To change download URLs, edit engines/manifest.json — no recompile needed.
1390
1391 use serde::{Deserialize, Serialize};
1392 use std::collections::HashMap;
1393 use std::path::PathBuf;
1394
1395 // -----
1396 // Data types
1397 // -----
1398
1399 #[derive(Debug, Clone, Serialize, Deserialize)]
1400 pub struct PlatformEntry {

```

```
1401     pub url:      String,
1402     /// hex SHA-256 of the downloaded archive
1403     pub sha256:    String,
1404     /// sub-directory name to extract into (inside components/<name>/ or goods/<id>/)
1405     pub extract_to: Option<String>,
1406     /// relative path to the main executable (used to set QDIR_<NAME> etc.)
1407     pub bin:       Option<String>,
1408     /// "zip" | "tar.gz" | "binary" (raw file, no archive)
1409     pub kind:      String,
1410 }
1411
1412 #[derive(Debug, Clone, Serialize, Deserialize)]
1413 pub struct ComponentManifest {
1414     pub name:      String,
1415     pub version:   String,
1416     pub platforms: HashMap<String, PlatformEntry>,
1417 }
1418
1419 // -----
1420 // Hardcoded component registry (MVP)
1421 // -----
1422
1423 /// Detect current platform key.
1424 pub fn current_platform() -> &'static str {
1425     #[cfg(all(target_os = "windows", target_arch = "x86_64"))] { "win32-x64" }
1426     #[cfg(all(target_os = "linux",   target_arch = "x86_64"))] { "linux-x64" }
1427     #[cfg(all(target_os = "macos",   target_arch = "x86_64"))] { "darwin-x64" }
1428     #[cfg(all(target_os = "macos",   target_arch = "aarch64"))] { "darwin-arm64" }
1429     #[cfg(not(any(
1430         all(target_os = "windows", target_arch = "x86_64"),
1431         all(target_os = "linux",   target_arch = "x86_64"),
1432         all(target_os = "macos",   target_arch = "x86_64"),
1433         all(target_os = "macos",   target_arch = "aarch64"),
1434     )))] { "unknown" }
1435 }
1436
1437 // -----
1438 // External manifest loading (JSON files, no recompile needed)
1439 // -----
1440
1441 #[derive(Debug, Clone, Serialize, Deserialize)]
1442 struct ManifestFile {
1443     #[serde(default)]
1444     _comment: String,
1445     #[serde(default)]
1446     _version: u32,
1447     components: Vec<ComponentManifest>,
1448 }
1449
1450 /// Try to load manifest from a JSON file.
```

```

1451 fn load_manifest_json(path: &std::path::Path) -> Option<Vec<ComponentManifest>> {
1452     let data = std::fs::read_to_string(path).ok()?;
1453     let mf: ManifestFile = serde_json::from_str(&data).ok()?;
1454     if mf.components.is_empty() {
1455         None
1456     } else {
1457         Some(mf.components)
1458     }
1459 }
1460
1461 /// Try to load external manifest from known search paths.
1462 fn load_external_manifest(qdir: Option<&PathBuf>) -> Option<Vec<ComponentManifest>> {
1463     // Path 1: user override — <QDIR>/userData/manifest.json
1464     if let Some(qd) = qdir {
1465         let user_manifest = qd.join("userData").join("manifest.json");
1466         if let Some(cmps) = load_manifest_json(&user_manifest) {
1467             eprintln!("[manifest] loaded user manifest: {}", user_manifest.display());
1468             return Some(cmps);
1469         }
1470     }
1471
1472     // Path 2: shipped manifest — next to ghrun.exe (engines/ directory)
1473     if let Ok(exe) = std::env::current_exe() {
1474         if let Some(exe_dir) = exe.parent() {
1475             let shipped = exe_dir.join("manifest.json");
1476             if let Some(cmps) = load_manifest_json(&shipped) {
1477                 eprintln!("[manifest] loaded shipped manifest: {}", shipped.display());
1478                 return Some(cmps);
1479             }
1480         }
1481     }
1482
1483     None
1484 }
1485
1486 // -----
1487 // Unified component lookup — tries external JSON first, then hardcoded
1488 // -----
1489
1490 /// Get all components, preferring external manifest over hardcoded.
1491 pub fn all_components(qdir: Option<&PathBuf>) -> Vec<ComponentManifest> {
1492     if let Some(ext) = load_external_manifest(qdir) {
1493         return ext;
1494     }
1495     builtin_components()
1496 }
1497
1498 /// Look up a component by name (with optional QDIR for external manifest).
1499 pub fn find_component_in(all_components: &[ComponentManifest], name: &str) -> Option<ComponentManifest>
1500     all_components.iter().find(|c| c.name == name).cloned()

```

```

98758 // ---- Wing panel constants (must match shell-wings.js _shAiW) ----
98759
98760 export const WING_WIDTH = 389;
98761
98762 export const CENTER_MIN_W = 1100;
98763
98764 export const CENTER_MIN_H = 800;
98765
98766
98767
98768 /** Update window minimum size based on which wings are open */
98769 export function updateWingMinSize(win: BrowserWindow, leftOpen: boolean, rightOpen: boolean): void {
98770     if (!win || win.isDestroyed()) return;
98771     const wingW = (leftOpen ? WING_WIDTH : 0) + (rightOpen ? WING_WIDTH : 0);
98772     win.setMinimumSize(CENTER_MIN_W + wingW, CENTER_MIN_H);
98773 }
98774
98775 // ---- Wing open/closed state — 窗口记忆, 与 bounds 同链双写 (2026-08-09) ----
98776 // ★ 翼开关状态从项目级 only.sql 升入窗口记忆: 双写 global.sql wings_bulbs + OS 级 ws.sql windowWings
98777 // 与 windowBounds 同一恢复链 → 重启后翼状态与窗口尺寸必然一致 (旧 only.sql 由 renderer 一次性迁移)
98778 const _windowWingMap = new Map<number, { left: boolean; right: boolean }>();
98779
98780 export function setWindowWingState(win: BrowserWindow, leftOpen: boolean, rightOpen: boolean, stateStore
98781     if (!win || win.isDestroyed()) return;
98782     const w = { left: !!leftOpen, right: !!rightOpen };
98783     _windowWingMap.set(win.id, w);
98784     updateWingMinSize(win, w.left, w.right);
98785     try {
98786         if (stateStore) stateStore.set('qqide', 'wings_bulbs', w).catch(() => {});
98787         wsStateSetKey('windowWings', w).catch(() => {});
98788     } catch { /* ignore */ }
98789 }
98790
98791 async function _readWingsFor(win: BrowserWindow, stateStore: StateStore): Promise<{ left: boolean; right
98792     // 多窗口还原时每窗口覆盖值优先 (openWindows 条目自带 wings)
98793     const ov = (win as any).__qqRestoreWings;
98794     if (ov && typeof ov === 'object') return { left: !!ov.left, right: !!ov.right };
98795     try {
98796         let v: any = null;
98797         try { v = await stateStore.get('qqide', 'wings_bulbs'); } catch { /* ignore */ }
98798         if (!v) { try { v = await wsStateGetKey('windowWings'); } catch { /* ignore */ } }
98799         if (v && typeof v === 'object') return { left: !!v.left, right: !!v.right };
98800     } catch { /* ignore */ }
98801     return { left: false, right: false };
98802 }
98803
98804 export function getWindowWingState(win: BrowserWindow): { left: boolean; right: boolean } {
98805     return _windowWingMap.get(win.id) || { left: false, right: false };
98806 }
98807

```

```

98808 function _pushWingsTo(win: BrowserWindow, w: { left: boolean; right: boolean }): void {
98809     if (!win || win.isDestroyed() || win.webContents.isDestroyed()) return;
98810     try { win.webContents.send('qqide:wing:restore', w); } catch { /* ignore */ }
98811 }
98812
98813
98814
98815 // ---- Window bounds ----
98816 /** 恢复链: 绿色包级 global.sq3 → OS 级 ws.sq3 → 默认 (2026-08-09: ws.sq3 OS 兜底, 删包不丢)
98817 * ★ 2026-08-09: 翼状态与 bounds 同源同链恢复——先恢复翼 (openWindows 每窗口覆盖 → global wings_bulbs →
98818 * 再恢复 bounds; 翼开而 bounds 不含翼宽 (旧数据) → 宽度钳到最小合理值, 防中间区拉伸。*/
98819 export async function restoreWindowBounds(win: BrowserWindow, stateStore: StateStore): Promise<void> {
98820     try {
98821         // ① 翼状态先恢复 (map + 最小尺寸 + 推送 renderer, bounds 恢复后 CSS 即时就位)
98822         const wings = await _readWingsFor(win, stateStore);
98823         _windowWingMap.set(win.id, wings);
98824         updateWingMinSize(win, wings.left, wings.right);
98825         _pushWingsTo(win, wings);
98826
98827         let v: any = null;
98828         try { v = await stateStore.get('qqide', 'window_bounds'); } catch { /* ignore */ }
98829         if (!v) { try { v = await wsStateGetKey('windowBounds'); } catch { /* ignore */ } }
98830         if (!v) { return; }
98831         if (v.maximized) {
98832             win.maximize();
98833             return;
98834         }
98835         if (typeof v.w === 'number' && typeof v.h === 'number' && v.w > 0 && v.h > 0) {
98836             let w = v.w;
98837             const wingW = (wings.left ? WING_WIDTH : 0) + (wings.right ? WING_WIDTH : 0);
98838             if (wingW > 0 && w < CENTER_MIN_W + wingW) w = CENTER_MIN_W + wingW;
98839             const displays = screen.getAllDisplays();
98840             const anyOverlap = displays.some(d => {
98841                 const dx = d.bounds.x, dy = d.bounds.y, dw = d.bounds.width, dh = d.bounds.height;
98842                 return (v.x < dx + dw && v.x + w > dx && v.y < dy + dh && v.y + v.h > dy);
98843             });
98844             if (anyOverlap) {
98845                 win.setBounds({ x: v.x || 0, y: v.y || 0, width: w, height: v.h });
98846             }
98847         }
98848     } catch (e) { /* ignore */ }
98849 }
98850
98851
98852
98853 // ---- Global keyboard shortcuts ----
98854
98855 export function registerGlobalKey(accel: string, id: string, mainWindow: BrowserWindow | null): boolean
98856
98857     try {

```



```

98858
98859     const ok = globalShortcut.register(accel, () => {
98860
98861         if (mainWindow && !mainWindow.isDestroyed()) {
98862
98863             try { mainWindow.webContents.send('qqide:key:global', { id, accel }); }
98864
98865             catch { /* ignore */ }
98866
98867         }
98868
98869     });
98870
98871     if (!ok) { console.warn('[key.global] register failed:', accel, id); }
98872
98873     return ok;
98874
98875 } catch (e) {
98876
98877     console.warn('[key.global] threw:', accel, id, e);
98878
98879     return false;
98880
98881 }
98882
98883 }
98884
98885
98886
98887 // ---- Window Project maps (for project lock) ----
98888 export const _windowProjectMap = new Map<number, string>();
98889 export const _projectWindowMap = new Map<string, number>();
98890
98891 // =====
98892 // ★ 窗口打开/关闭顺序记录 (2026-08-16 客户 bug 根治: 无法识别窗口关闭顺序)
98893 // 数据模型 (双写 global.sq3 + OS 级 ws.sq3):
98894 //     openWindows / open_windows      — 当前存活窗口快照 (打开序; 每次开窗/关窗即时落盘,
98895 //                                     崩溃/被杀兜底 = 最近一次快照; 恢复集唯一权威)
98896 //     windowOpenLog / window_open_log — 打开序日志 (有界 FIFO, 审计用)
98897 //     windowCloseLog / window_close_log — 关闭序日志 (mode='x' 单窗 / 'quitAll' 整组批次,
98898 //                                     批次携带完整成员列表, 有界 FIFO)
98899 // 核心语义:
98900 //     ① 单窗 X 关闭 → 快照收缩为「剩余存活窗」; 最后一个 → 含自身快照
98901 //         (重启只回到最后留存的那一个窗口; 之前关闭的窗口只留在 recents hover 列表)
98902 //     ② 菜单退出 (quitAll) → 整组快照在 quitAll 瞬间写死, 后续 close 事件绝不收缩
98903 //         (重启打开整组窗口, 两两间隔 500ms 防竞争态)
98904 //     ③ 半销毁窗口 (_closingWinIds) 绝不入快照 — 快速连关时已关窗口绝不复活
98905 // =====
98906 const _WINDOW_LOG_MAX = 50;
98907 const _windowOpenedAt = new Map<number, number>();           // winId → 打开时间戳

```

```

98908 const _windowOpenLog: Array<{ mainFolder: string; openedAt: number }> = [];
98909 const _windowCloseLog: Array<{ mainFolder: string; closedAt: number; mode: 'x' | 'quitAll'; batchId?: st
98910 const _windowOpenLogged = new Map<number, string>();          // winId → folder (打开日志等)
98911 const _closingWinIds = new Set<number>();                    // close 已放行、closed 未到 (半销毁)
98912 let _quitAllBatch: { batchId: string; at: number; members: any[] } | null = null;
98913
98914 function _normFolder(p: string): string {
98915     return (p || '').replace(/\\\/g, '/').replace(/\/$/g, '');
98916 }
98917
98918 function _persistOpenWindows(list: any[], stateStore: StateStore): void {
98919     try {
98920         stateStore.setNow('qqide', 'open_windows', list).catch(() => {});
98921         wsStateSetKey('openWindows', list).catch(() => {});
98922     } catch { /* ignore */ }
98923 }
98924
98925 function _persistWindowLogs(stateStore: StateStore): void {
98926     try {
98927         stateStore.setNow('qqide', 'window_open_log', _windowOpenLog.slice()).catch(() => {});
98928         stateStore.setNow('qqide', 'window_close_log', _windowCloseLog.slice()).catch(() => {});
98929         wsStateSetKey('windowOpenLog', _windowOpenLog.slice()).catch(() => {});
98930         wsStateSetKey('windowCloseLog', _windowCloseLog.slice()).catch(() => {});
98931     } catch { /* ignore */ }
98932 }
98933
98934 function _snapshotOf(win: BrowserWindow, folder: string): any | null {
98935     try {
98936         const n = _normFolder(folder);
98937         if (!n) return null;
98938         const b = win.getBounds();
98939         return {
98940             mainFolder: n,
98941             bounds: { x: b.x, y: b.y, w: b.width, h: b.height, maximized: win.isMaximized() },
98942             wings: _windowWingMap.get(win.id) || { left: false, right: false },
98943             openedAt: _windowOpenedAt.get(win.id) || Date.now(),
98944         };
98945     } catch { return null; }
98946 }
98947
98948 /** 存活窗口快照 (打开序 = _windowProjectMap 插入序; 半销毁/已销毁窗口绝不存) */
98949 function _snapshotOpenWindows(excludeWinId?: number): any[] {
98950     const list: any[] = [];
98951     const seen = new Set<string>();
98952     for (const [winId, folder] of _windowProjectMap) {
98953         if (winId === excludeWinId) continue;
98954         if (_closingWinIds.has(winId)) continue;
98955         const win = BrowserWindow.fromId(winId);
98956         if (!win || win.isDestroyed()) continue;
98957         const snap = _snapshotOf(win, folder);

```

```

98958         if (!snap || seen.has(snap.mainFolder)) continue;
98959         seen.add(snap.mainFolder);
98960         list.push(snap);
98961     }
98962     return list;
98963 }
98964
98965 export function snapshotOpenWindows(): any[] {
98966     return _snapshotOpenWindows();
98967 }
98968
98969 /** ★ 窗口项目注册时记录打开序 + 刷新存活快照（崩溃兜底：快照永远最新）*/
98970 export function recordWindowOpen(winId: number, folder: string, stateStore: StateStore): void {
98971     try {
98972         const n = _normFolder(folder);
98973         if (!n) return;
98974         if (_windowOpenLogged.get(winId) === n) return; // 幂等：同窗同项目只记一次
98975         _windowOpenLogged.set(winId, n);
98976         _windowOpenedAt.set(winId, Date.now());
98977         _windowOpenLog.push({ mainFolder: n, openedAt: _windowOpenedAt.get(winId)! });
98978         if (_windowOpenLog.length > _WINDOW_LOG_MAX) _windowOpenLog.shift();
98979         _persistWindowLogs(stateStore);
98980         const list = _snapshotOpenWindows();
98981         if (list.length > 0) _persistOpenWindows(list, stateStore);
98982     } catch (e: any) { console.warn('[window-manager] recordWindowOpen failed:', e && e.message); }
98983 }
98984
98985 /** ★ 菜单退出整组关闭: quitAll 瞬间捕获全组快照写死, 防 close 事件逐步收缩 (2026-08-16) */
98986 export function beginQuitAllBatch(stateStore: StateStore): void {
98987     try {
98988         if (_quitAllBatch) return; // 幂等
98989         const members: any[] = [];
98990         for (const [winId, folder] of _windowProjectMap) {
98991             if (_closingWinIds.has(winId)) continue;
98992             const win = BrowserWindow.fromId(winId);
98993             if (!win || win.isDestroyed()) continue;
98994             const snap = _snapshotOf(win, folder);
98995             if (!snap) continue;
98996             snap.winId = winId;
98997             members.push(snap);
98998         }
98999         if (members.length === 0) return;
99000         _quitAllBatch = { batchId: Date.now() + '-' + Math.random().toString(36).slice(2, 8), at: Date.now() };
99001         // ★ 整组快照即时落盘（不等 before-quit — 退出路径千变万化，这里先写死）
99002         const list = members.map((m: any) => ({ mainFolder: m.mainFolder, bounds: m.bounds, wings: m.win
99003         _persistOpenWindows(list, stateStore);
99004         // 关闭序日志：整组批次（成员列表随记录，关闭顺序一目了然）
99005         _windowCloseLog.push({ mainFolder: '', closedAt: Date.now(), mode: 'quitAll', batchId: _quitAllB
99006         if (_windowCloseLog.length > _WINDOW_LOG_MAX) _windowCloseLog.shift();
99007         _persistWindowLogs(stateStore);

```

```
99008     console.log(' [window-manager] quitAll batch captured: ' + members.length + ' window(s)');
99009   } catch (e: any) { console.warn(' [window-manager] beginQuitAllBatch failed:', e && e.message); }
99010 }
99011
99012 /** ★ 关闭时立即保存剩余窗口列表 (2026-08-09 初版 / 2026-08-16 关闭序重构):
99013  * 剩余窗口活着 → 列表=剩余(打开序); 最后一个窗口关闭 → 列表=自身(退出后还原);
99014  * 菜单退出批次 → 保留整组快照绝不收缩; 双写 global.sq3 open_windows + 0S 级 ws.sq3 openWindows */
99015 function _saveOpenWindowsNow(closingWin: BrowserWindow, stateStore: StateStore): void {
99016   try {
99017     _closingWinIds.add(closingWin.id);
99018     // ★ 菜单退出批次: 整组快照保留 (批次成员逐窗 close 事件全部写同一完整列表, 幂等)
99019     if (_quitAllBatch && _quitAllBatch.members.some((m: any) => m.winId === closingWin.id)) {
99020       const list = _quitAllBatch.members
99021         .filter((m: any) => m.mainFolder)
99022         .map((m: any) => ({ mainFolder: m.mainFolder, bounds: m.bounds, wings: m.wings, openedAt
99023           if (list.length > 0) _persistOpenWindows(list, stateStore);
99024       return; // 批次关闭不记单窗日志 (批记录已含完整成员列表)
99025     }
99026     // ★ 单窗 X 关闭: 存活快照 = 打开序 - 本窗 - 半销毁窗
99027     let list = _snapshotOpenWindows(closingWin.id);
99028     const self = _snapshotOf(closingWin, _windowProjectMap.get(closingWin.id) || '');
99029     if (list.length === 0 && self) list.push(self); // 最后一个窗口 → 含自身快照
99030     // 关闭序日志 (仅记录带项目的窗口)
99031     const folder = _windowProjectMap.get(closingWin.id);
99032     if (folder) {
99033       const n = _normFolder(folder);
99034       if (n) {
99035         _windowCloseLog.push({ mainFolder: n, closedAt: Date.now(), mode: 'x' });
99036         if (_windowCloseLog.length > _WINDOW_LOG_MAX) _windowCloseLog.shift();
99037         _persistWindowLogs(stateStore);
99038       }
99039     }
99040     // ★ 空列表也写 (清陈旧恢复集 — 最后一个窗口无项目时, 绝不复活旧窗口)
99041     _persistOpenWindows(list, stateStore);
99042   } catch { /* ignore */ }
99043 }
99044
99045
99046
99047 // ★ 关闭确认旁路: 菜单退出时设置, 跳过 Alt+F4 确认框
99048
99049 export function bypassCloseConfirm(win: BrowserWindow): void {
99050
99051   (win as any).__qqCloseBypass = true;
99052
99053 }
99054
99055
99056
99057 // ---- createWindow ----
```

```
99058
99059 export function createWindow(
99060
99061     portableRoot: string,
99062
99063     portableCache: string,
99064
99065     appVersion: string,
99066
99067     // lspBridge: LspBridge, // LSP OFF — 2026-06-23
99068
99069     lspBridge: any,
99070
99071     downloadService: DownloadService,
99072
99073     stateStore: StateStore,
99074
99075 ): BrowserWindow {
99076
99077     const preloadPath = path.join(__dirname, 'preload.js');
99078
99079     const win = new BrowserWindow({
99080
99081         width: 1400,
99082
99083         height: 900,
99084
99085         minWidth: CENTER_MIN_W,
99086
99087         minHeight: CENTER_MIN_H,
99088
99089         show: false,
99090
99091         frame: false,
99092
99093         backgroundColor: '#fdf6e3',
99094
99095         title: 'qqide',
99096
99097         webPreferences: {
99098             preload: preloadPath,
99099             contextIsolation: true,
99100             nodeIntegration: false,
99101             sandbox: false,
99102             webSecurity: false,
99103             spellcheck: false,
99104             additionalArguments: [
99105                 `--qqide-root=${portableRoot}`,
99106                 `--qqide-version=${appVersion}`,
99107             ],
```

```
99108     },
99109   });
99110
99111   // ★ 窗口编队认领: 创建即按序认领最近空闲槽位 (1 2 q w a s z x), 无空闲=null (>8窗口)
99112   claimSquad(win);
99113   // ★ 新窗口认领 → 广播 (他窗下拉/按钮秒同步)
99114   broadcastSquadState();
99115
99116   // ★ console-message 始终运行 — 捕获全量消息 (CDP 不再阻塞, 仅作补充)
99117
99118   const _cmHandler = (
99119
99120     _e: any, _level: number, message: string, _line: number, _sourceId: string
99121
99122   ) => {
99123
99124     // _level 0=verbose → 过滤 (DevTools save 不显示 verbose)
99125
99126     if (_level === 0) return;
99127
99128     const src = (_sourceId || '').replace(/\\/g, '/');
99129
99130     const file = src.split('/').pop() || '';
99131
99132     const prefix = file && !file.startsWith('(') && _line ? file + ':' + _line + ' ' : '';
99133
99134     _consoleBuffer.push(prefix + message);
99135
99136     if (_consoleBuffer.length > _consoleMaxLines) _consoleBuffer.shift();
99137
99138   };
99139
99140   win.webContents.on('console-message', _cmHandler);
99141
99142
99143
99144   // ★ CDP 作为补充 (网络错误调用栈等) — 静默失败不影响 console-message 主线
99145
99146   _setupCdpConsoleCapture(win).catch(() => {});
99147
99148
99149
99150   win.removeMenu();
99151
99152
99153
99154   win.once('ready-to-show', async () => {
99155
99156     await restoreWindowBounds(win, stateStore);
99157
```

```

99158      // ★ 不在 ready-to-show 就 show() — 等 boot 完成加载面板到 100% 才由 boot.ts 调用 show()
99159
99160    });
99161
99162
99163
99164    // ★ 关闭确认: Alt+F4 / 右上角 X → 通知 renderer 弹自定义确认框; 菜单退出 → 跳过
99165    win.on('close', (e) => {
99166      if ((win as any).__qqCloseBypass) {
99167        // ★ 窗口即将销毁 (X 关闭/确认退出路径) — 立即捕获最终 bounds + 窗口列表 (2026-08-09 修复:
99168        // 旧实现依赖 before-quit saveAllOpenWindows, 但 X 关闭时窗口已销毁 → open_windows 永不保
99169        try {
99170          const b = win.getBounds();
99171          const obj = { x: b.x, y: b.y, w: b.width, h: b.height, maximized: win.isMaximized() };
99172          stateStore.setNow('qqide', 'window_bounds', obj).catch(() => {});
99173          wsStateSetKey('windowBounds', obj).catch(() => {});
99174          // ★ 翼状态与 bounds 同步落盘 (最后关闭瞬间的真实翼状态)
99175          const wings = _windowWingMap.get(win.id) || { left: false, right: false };
99176          stateStore.setNow('qqide', 'wings_bulbs', wings).catch(() => {});
99177          wsStateSetKey('windowWings', wings).catch(() => {});
99178          _saveOpenWindowsNow(win, stateStore);
99179        } catch { /* ignore */ }
99180        return; // 菜单退出已设旁路, 直接放行
99181      }
99182      e.preventDefault();
99183      // ★ 武装键盘: 确认框弹出期间 Enter=确认 / Escape=取消
99184      (win as any).__qqConfirmArmed = true;
99185      // 通知 renderer 弹出确认框 (renderer 样式同设置面板)
99186      if (!win.isDestroyed() && !win.webContents.isDestroyed()) {
99187        try { win.webContents.send('qqide:confirm-close'); } catch {}
99188      }
99189    });
99190
99191    // ★ 确认框键盘键入 (Enter=确认退出 / Escape=取消) — before-input-event 在浏览器进程捕获,
99192    // 焦点在左中右面板 iframe 内 (document keydown 收不到) 也能 100% 响应
99193    win.webContents.on('before-input-event', (event: any, input: any) => {
99194      if (!(win as any).__qqConfirmArmed) { return; }
99195      if (input.type !== 'keyDown' && input.type !== 'rawKeyDown') { return; }
99196      if (input.isComposing) { return; } // IME 组词回车不触发
99197      if (input.key === 'Enter') {
99198        event.preventDefault();
99199        (win as any).__qqConfirmArmed = false;
99200        bypassCloseConfirm(win);
99201        // ★ 立即隐藏确认框 (force 越过 1s 防抖): 确认退出永不防抖, 窗口马上开始关闭
99202        try { win.webContents.send('qqide:confirm-close-dismiss', true); } catch { /* ignore */ }
99203        try { win.close(); } catch { /* ignore */ }
99204      } else if (input.key === 'Escape') {
99205        event.preventDefault();
99206        try { win.webContents.send('qqide:confirm-close-dismiss'); } catch { /* ignore */ }
99207      }

```

```

99208     });
99209
99210     // ★ 渲染进程崩溃监控 + 自动恢复 (2026-08-08 F13):
99211     // 崩溃(含 V8 OOM) → 记录 reason + exitCode 到 Data/alphal/render-crash.log → 防抖 3s 自动 reload
99212     // 窗口不消失; 下次崩溃即可凭 reason 实锤根因 ('oom' = V8 堆耗尽)
99213     const _crashLogPath = path.join(portableRoot, 'Data', 'alphal', 'render-crash.log');
99214     win.webContents.on('render-process-gone', (_e, details) => {
99215         const reason = (details && details.reason) || 'unknown';
99216         try {
99217             fs.mkdirSync(path.dirname(_crashLogPath), { recursive: true });
99218             fs.appendFileSync(_crashLogPath,
99219                 new Date().toISOString() + ' reason=' + reason +
99220                 ' exitCode=' + ((details && details.exitCode) || 0) +
99221                 ' win=' + win.id + '\n');
99222         } catch (_) { /* ignore */ }
99223         try { console.error('[window-manager] render-process-gone reason=' + reason + ' win=' + win.id);
99224             // ★ 天罗地网: 崩溃事件 + 全量快照 (主进程此刻仍存活, 必须立即落盘)
99225             try { crashNetLog({ kind: 'render-gone', winId: win.id, reason, exitCode: (details && details.ex
99226                 try { crashNetSnapshot('render-gone'); } catch (_) { }
99227             if (reason === 'clean-exit') { return; } // 正常关闭路径, 不恢复
99228             // 自动恢复: 崩溃/被杀/OOM → reload 保留窗口 (防抖 3s, 防崩溃循环风暴)
99229             const now = Date.now();
99230             const lastAt = (win as any).__qqqCrashReloadAt || 0;
99231             if (now - lastAt > 3000) {
99232                 (win as any).__qqqCrashReloadAt = now;
99233                 setTimeout(() => {
99234                     try { if (!win.isDestroyed() && !win.webContents.isDestroyed()) { win.webContents.reload
99235                         }, 300);
99236                 }
99237             });
99238
99239             // 渲染进程卡死(JS 长任务/GC 停顿) — 只记录, 不杀(杀会丢未落盘数据)
99240             win.webContents.on('unresponsive', () => {
99241                 try { console.error('[window-manager] renderer unresponsive win=' + win.id); } catch (_) { }
99242             });
99243
99244             win.on('closed', () => {
99245                 (win as any).__qqqConfirmArmed = false;
99246                 // ★ 半销毁标记清除 (2026-08-16): 窗口彻底销毁, 不再污染存活快照
99247                 _closingWinIds.delete(win.id);
99248                 if (_quitAllBatch) {
99249                     // 批次成员全部销毁 → 批次状态清空 (进程即将退出, 防陈旧状态污染)
99250                     const alive = _quitAllBatch.members.some((m: any) => {
99251                         const w = BrowserWindow.fromId(m.winId);
99252                         return w && !w.isDestroyed() && !_closingWinIds.has(m.winId);
99253                     });
99254                     if (!alive) _quitAllBatch = null;
99255                 }
99256                 try { if (_boundsSaveTimer) { clearTimeout(_boundsSaveTimer); _boundsSaveTimer = null; } } catch
99257                 // ★ 无条件释放项目锁 (2026-08-13 幽灵锁事故根因之一): projectLock.claim (only-store)

```



```

99258      // 与 window.claimProject (注册 _windowProjectMap) 是两条独立 IPC——时序竞态下
99259      //  claim 已成功但 map 未注册 → 旧逻辑条件释放被跳过 → _held 残留 + 心跳永续
99260      //  → 幽灵锁 (绿色包误报 pid=9424=自己 实锤)。releaseProject 内部 _held 无条目时安全 return。
99261      try { releaseProject(win.id); } catch (_) {}
99262      // ★ 窗口关闭 → 编队槽位回到空闲 + 广播 (他窗秒同步)
99263      try { releaseSquad(win.id); } catch (_) {}
99264      try { broadcastSquadState(); } catch (_) {}
99265      try {
99266          const ownedProject = _windowProjectMap.get(win.id);
99267          if (ownedProject) {
99268              _windowProjectMap.delete(win.id);
99269              _projectWindowMap.delete(ownedProject);
99270          }
99271      } catch (_) {}
99272  });
99273
99274
99275
99276      // Window blur → dismiss AI viewport dropdown
99277
99278      win.on('blur', () => {
99279
99280          if (!win.isDestroyed() && !win.webContents.isDestroyed()) {
99281
99282              win.webContents.executeJavaScript(
99283
99284                  `if(window.qqideViewport&&window.qqideViewport.closeDropdown)window.qqideViewport.clo
99285
99286                  ).catch((e) => { try { console.warn('[main] blur dismiss failed:', e && e.message); } catch
99287
99288              }
99289
99290      });
99291
99292
99293
99294      // Persist window bounds on resize/move (debounced 500ms)
99295      let _boundsSaveTimer: NodeJS.Timeout | null = null;
99296      // ★ 双写: 绿色包级 global.sq3 + OS 级 ws.sq3 (删包/换包后 OS 兜底回写)
99297      const persistBounds = (b: Electron.Rectangle, maximized: boolean) => {
99298          const obj = { x: b.x, y: b.y, w: b.width, h: b.height, maximized };
99299          stateStore.set('qqide', 'window_bounds', obj).catch(() => {});
99300          wsStateSetKey('windowBounds', obj).catch(() => {});
99301      };
99302      const saveBounds = () => {
99303          try {
99304              if (win.isDestroyed() || win.isMinimized() || win.isMaximized()) { return; }
99305              persistBounds(win.getBounds(), false);
99306          } catch { /* ignore */ }
99307      };

```

```
99308     const debouncedSaveBounds = () => {
99309         if (_boundsSaveTimer) { clearTimeout(_boundsSaveTimer); }
99310         _boundsSaveTimer = setTimeout(saveBounds, 500);
99311     };
99312     win.on('resize', debouncedSaveBounds);
99313     win.on('move', debouncedSaveBounds);
99314     win.on('maximize', () => {
99315         try {
99316             stateStore.set('qqide', 'window_bounds', { maximized: true }).catch(() => { });
99317             wsStateSetKey('windowBounds', { maximized: true }).catch(() => { });
99318         } catch { /* ignore */ }
99319     });
99320     win.on('unmaximize', () => { try { saveBounds(); } catch { /* ignore */ } });
99321
99322
99323
99324     // Download progress
99325
99326     downloadService.setProgressSender((entry) => {
99327
99328         if (win && !win.isDestroyed()) {
99329
99330             try { win.webContents.send('qqide:download:progress', entry); } catch { /* ignore */ }
99331
99332         }
99333
99334     });
99335
99336
99337
99338     // Lock window UI at 1.0 (no zoom — editor font size handles text scaling)
99339
99340     win.webContents.on('did-finish-load', () => {
99341         win.webContents.setZoomFactor(1.0);
99342         // ★ 翼状态重推 (Ctrl+R 热重载后 renderer 需重新应用; 主进程 map 仍在)
99343         _pushWingsTo(win, _windowWingMap.get(win.id) || { left: false, right: false });
99344     });
99345
99346
99347
99348     // Ctrl/Cmd + (+/-/0) editor font size shortcuts
99349
99350     win.webContents.on('before-input-event', (ev, input) => {
99351
99352         if (input.type !== 'keyDown') { return; }
99353
99354         const ctrl = input.control || input.meta;
99355
99356         if (!ctrl) { return; }
99357
```

```
99358     const k = input.key;
99359
99360     if (k === '=' || k === '+') {
99361
99362         ev.preventDefault();
99363
99364         editorFontSize = Math.min(128, editorFontSize + 1);
99365
99366         saveEditorFontSize(stateStore);
99367
99368         broadcastEditorFontSize(editorFontSize);
99369
99370     } else if (k === '-' || k === '_') {
99371
99372         ev.preventDefault();
99373
99374         editorFontSize = Math.max(1, editorFontSize - 1);
99375
99376         saveEditorFontSize(stateStore);
99377
99378         broadcastEditorFontSize(editorFontSize);
99379
99380     } else if (k === '0') {
99381
99382         ev.preventDefault();
99383
99384         editorFontSize = 13;
99385
99386         saveEditorFontSize(stateStore);
99387
99388         broadcastEditorFontSize(editorFontSize);
99389
99390     }
99391
99392 });
99393
99394
99395
99396 // ★ 控制台按钮注入 — 无论 dev/prod, DevTools 打开就注入
99397
99398 win.webContents.on('devtools-opened', () => {
99399
99400     const dwc = (win.webContents as any).devToolsWebContents as WebContents;
99401
99402     if (dwc && !dwc.isDestroyed()) {
99403
99404         injectDevToolsConsoleButtons(dwc, win.webContents, () => _consoleBuffer.join('\n'), win);
99405
99406     }
99407
```

```
99408     });
99409
99410
99411
99412     // Dev mode extras
99413
99414     if (extractFlags().isDev) {
99415
99416         win.webContents.on('devtools-opened', () => {
99417             // ★ DevTools 窗口标题 → Python broker (跨平台: Win ctypes / Mac osascript / Linux wmcctl)
99418             // ★ 2026-08-23: 成功后即停 — 旧实现 renamed 成功仍打满 8 次重试 (devtools 窗口已存在时
99419             //   EnumWindows 扫不到 → 8 条 FAILED 日志噪音); devtools 重开触发新闭包重新循环, 覆盖不丢
99420             let _renameDone = false;
99421             const _doRename = async (attempt: number) => {
99422                 const p = _windowProjectMap.get(win.id);
99423                 const n = p ? path.basename(p) : 'qqide';
99424                 console.log('[devtools] rename attempt=' + attempt + ' winId=' + win.id + ' projPath=' +
99425                     const ok = await renameDevToolsViaBroker(win, n);
99426                 if (ok) _renameDone = true;
99427             };
99428             for (let _ri = 1; _ri <= 8; _ri++) {
99429                 setTimeout(() => { if (!_renameDone) _doRename(_ri); }, 1500 * _ri);
99430             }
99431         });
99432
99433         win.webContents.openDevTools({ mode: 'detach' });
99434
99435         win.webContents.on('before-input-event', (ev, input) => {
99436
99437             if (input.type !== 'keyDown') { return; }
99438
99439             if (input.key === 'F5' || (input.control && input.key.toLowerCase() === 'r')) {
99440
99441                 ev.preventDefault();
99442
99443                 win.webContents.reloadIgnoringCache();
99444
99445             }
99446
99447             if (input.control && input.shift && input.key.toLowerCase() === 'i') {
99448
99449                 ev.preventDefault();
99450
99451                 if (win.webContents.isDevToolsOpened()) {
99452
99453                     win.webContents.closeDevTools();
99454
99455                 } else {
99456
99457                     win.webContents.openDevTools({ mode: 'detach' });
```

```
99458
99459     }
99460
99461     }
99462
99463     // ★ 吃 F12: Chromium 默认 F12=DevTools, IDE 自己不用 F12, 阻止窗口被劫持
99464
99465     if (input.key === 'F12') {
99466
99467         ev.preventDefault();
99468
99469     }
99470
99471     });
99472
99473     }
99474
99475
99476
99477     return win;
99478
99479 }
99480
99481
99482
99483 // —— CDP 控制台全量捕获 —— Log.entryAdded = DevTools 另存为 100% 同源数据 ——
99484
99485 // ★ v8: 延迟启动 + 重试 + 完整覆盖
99486
99487 async function _setupCdpConsoleCapture(win: BrowserWindow): Promise<void> {
99488
99489     const PORT = 8315;
99490     const allSockets: SimpleWebSocket[] = [];
99491     let _started = false;
99492
99493     // Safe logger: console.log → EPIPE when stdout is broken (no console window)
99494     const _safeLog = (...args: any[]) => { try { console.log(...args); } catch {} };
99495
99496     // DevTools Console 默认不显示的 source (intervention=性能建议, rendering=渲染, violation=违规, depr
99497
99498     const _NOISE_SOURCES = new Set(['intervention', 'rendering', 'violation', 'deprecation', 'recommenda
99499
99500
99501
99502     const _handleEntry = (entry: any) => {
99503
99504         // 跳过 DevTools Console 默认不显示的条目
99505
99506         if (!entry || !entry.text) return;
99507
```

```
99508         if (entry.level === 'verbose') return;
99509
99510         if (entry.source && _NOISE_SOURCES.has(entry.source)) return;
99511
99512         // Log 域 network 条目: 用 entry.url 补全文本
99513
99514         // Console 域不覆盖网络错误 → Log 域是唯一来源
99515
99516         if (entry.source === 'network') {
99517
99518             const nUrl = (entry.url || '').replace(/\\\/g, '/');
99519
99520             const nFile = nUrl.split('/').pop() || nUrl;
99521
99522             const nText = (entry.text || '');
99523
99524             const nCallFrames: any[] = entry.stackTrace?.callFrames || [];
99525
99526             const nLines: string[] = [];
99527
99528             if (nCallFrames.length > 0) {
99529
99530                 const f0 = nCallFrames[0];
99531
99532                 const fUrl = ((f0.url || '').replace(/\\\/g, '/').split('/').pop() || f0.url);
99533
99534                 const fLine = f0.lineNumber || 0;
99535
99536                 nLines.push((fUrl && fLine ? fUrl + ':' + fLine + ' ' : '') + nUrl + ' ' + nText);
99537
99538                 for (let i = 1; i < nCallFrames.length; i++) {
99539
99540                     const cf = nCallFrames[i];
99541
99542                     const fn = cf.functionName || '<anonymous>';
99543
99544                     const fu = ((cf.url || '').replace(/\\\/g, '/').split('/').pop() || cf.url);
99545
99546                     nLines.push('    ' + fn + ' @ ' + fu + ':' + (cf.lineNumber || 0));
99547
99548                 }
99549
99550             } else {
99551
99552                 nLines.push(nUrl + ' ' + nText);
99553
99554             }
99555
99556             for (const l of nLines) {
99557
```

```
99558         _consoleBuffer.push(l);
99559
99560         if (_consoleBuffer.length > _consoleMaxLines) _consoleBuffer.shift();
99561
99562     }
99563
99564     return;
99565
99566 }
99567
99568
99569
99570     const url = (entry.url || '').replace(/\\/g, '/');
99571
99572     const file = url.split('/').pop() || url;
99573
99574     const line = entry.lineNumber || 0;
99575
99576     const text = entry.text || '';
99577
99578     const callFrames: any[] = entry.stackTrace?.callFrames || [];
99579
99580
99581
99582     const lines: string[] = [];
99583
99584     if (callFrames.length > 0) {
99585
99586         const f0 = callFrames[0];
99587
99588         const fUrl = ((f0.url || '').replace(/\\/g, '/').split('/').pop() || f0.url);
99589
99590         const fLine = f0.lineNumber || 0;
99591
99592         lines.push((fUrl && fLine ? fUrl + ':' + fLine + ' ' : '') + text);
99593
99594         for (let i = 1; i < callFrames.length; i++) {
99595
99596             const cf = callFrames[i];
99597
99598             const fn = cf.functionName || '<anonymous>';
99599
99600             const fu = ((cf.url || '').replace(/\\/g, '/').split('/').pop() || cf.url);
99601
99602             lines.push('    ' + fn + ' @ ' + fu + ':' + (cf.lineNumber || 0));
99603
99604         }
99605
99606     } else if (file && line) {
99607
```

```
99608         lines.push(file + ':' + line + ' ' + text);
99609
99610     } else {
99611
99612         lines.push(text);
99613
99614     }
99615
99616     for (const l of lines) {
99617
99618         _consoleBuffer.push(l);
99619
99620         if (_consoleBuffer.length > _consoleMaxLines) _consoleBuffer.shift();
99621
99622     }
99623
99624 };
99625
99626
99627
99628 // 去重 set: Console message 的 url+line+text 前80字符 作为 dedup key
99629
99630 const _consoleDedup = new Set<string>();
99631
99632
99633
99634 const _handleConsoleMsg = (msg: any) => {
99635
99636     if (!msg || !msg.text) return;
99637
99638     if (msg.level === 'verbose') return;
99639
99640     if (msg.source && _NOISE_SOURCES.has(msg.source)) return;
99641
99642     // Console 域 network 消息无 URL — 由 Log 域 _handleEntry 处理
99643
99644     if (msg.source === 'network') return;
99645
99646     const url = (msg.url || '').replace(/\\\/g, '/');
99647
99648     const file = url.split('/').pop() || url;
99649
99650     const line = msg.line || 0;
99651
99652     const text = msg.text || '';
99653
99654     const callFrames: any[] = msg.stackTrace?.callFrames || [];
99655
99656
99657
```



```
99658      // Dedup: same file+line+text prefix within 1s
99659
99660      const dedupKey = file + ':' + line + ':' + text.slice(0, 80);
99661
99662      if (!_consoleDedup.has(dedupKey)) return;
99663
99664      _consoleDedup.add(dedupKey);
99665
99666      setTimeout(() => _consoleDedup.delete(dedupKey), 1000);
99667
99668
99669
99670      const lines: string[] = [];
99671
99672      if (callFrames.length > 0) {
99673
99674          const f0 = callFrames[0];
99675
99676          const fUrl = ((f0.url || '').replace(/\\/g, '/').split('/').pop() || f0.url);
99677
99678          const fLine = f0.lineNumber || 0;
99679
99680          lines.push((fUrl && fLine ? fUrl + ':' + fLine + ' ' : '') + text);
99681
99682          for (let i = 1; i < callFrames.length; i++) {
99683
99684              const cf = callFrames[i];
99685
99686              const fn = cf.functionName || '<anonymous>';
99687
99688              const fu = ((cf.url || '').replace(/\\/g, '/').split('/').pop() || cf.url);
99689
99690              lines.push('    ' + fn + ' @ ' + fu + ':' + (cf.lineNumber || 0));
99691
99692          }
99693
99694      } else if (file && line) {
99695
99696          lines.push(file + ':' + line + ' ' + text);
99697
99698      } else {
99699
99700          lines.push(text);
99701
99702      }
99703
99704      for (const l of lines) {
99705
99706          _consoleBuffer.push(l);
99707
```

```
99708         if (_consoleBuffer.length > _consoleMaxLines) _consoleBuffer.shift();
99709
99710     }
99711
99712 };
99713
99714
99715
99716 const _connectTarget = (wsUrl: string, label: string) => {
99717
99718     try {
99719
99720         const ws = new SimpleWebSocket(wsUrl);
99721
99722         allSockets.push(ws);
99723
99724         ws.on('open', () => {
99725
99726             // ★ Console 域 → 完全格式化消息 (含网络错误的 GET URL net::ERR_*)
99727
99728             ws.send(JSON.stringify({ id: 1, method: 'Console.enable', params: {} }));
99729
99730             // ★ Log 域 → 原始日志引擎消息 (含栈帧, 兜底)
99731
99732             ws.send(JSON.stringify({ id: 2, method: 'Log.enable', params: {} }));
99733
99734             _safeLog('[main] CDP Console+Log enabled for ' + label);
99735
99736         });
99737
99738         ws.on('message', (data: string) => {
99739
99740             try {
99741
99742                 const msg = JSON.parse(data);
99743
99744                 if (msg.method === 'Console.messageAdded' && msg.params?.message) {
99745
99746                     _handleConsoleMsg(msg.params.message);
99747
99748                 } else if (msg.method === 'Log.entryAdded' && msg.params?.entry) {
99749
99750                     _handleEntry(msg.params.entry);
99751
99752                 }
99753
99754             } catch {}
99755
99756         });
99757
```

```
99758         ws.on('error', (err: Error) => {
99759
99760             _safeLog('[main] CDP WS error (' + label + ')', err.message);
99761
99762         });
99763
99764     } catch (e: any) {
99765
99766         _safeLog('[main] CDP connect failed (' + label + ')', e.message);    }
99767
99768     };
99769
99770
99771
99772     const _doCapture = async () => {
99773
99774         if (!_started) return;
99775
99776         try {
99777
99778             const targetsJson: string = await new Promise<string>((resolve, reject) => {
99779
99780                 const req = http.get(`http://127.0.0.1:${PORT}/json/list`, (res) => {
99781
99782                     let data = '';
99783
99784                     res.on('data', (chunk: string) => data += chunk);
99785
99786                     res.on('end', () => resolve(data));
99787
99788                 });
99789
99790                 req.on('error', reject);
99791
99792                 req.setTimeout(3000, () => { req.destroy(); reject(new Error('timeout')); });
99793
99794             });
99795
99796             const targets: any[] = JSON.parse(targetsJson);
99797
99798             let pageCount = 0;
99799
99800             for (const t of targets) {
99801
99802                 if (t.type === 'page' && t.webSocketDebuggerUrl) {
99803
99804                     _connectTarget(t.webSocketDebuggerUrl, 'page:' + (t.url || '').slice(0, 60));
99805
99806                     pageCount++;
99807
```

```
99808         }
99809
99810     }
99811
99812     // 浏览器级 CDP
99813
99814     try {
99815
99816         const versionJson: string = await new Promise<string>((resolve, reject) => {
99817
99818             const req = http.get(`http://127.0.0.1:${PORT}/json/version`, (res) => {
99819
99820                 let data = '';
99821
99822                 res.on('data', (chunk: string) => data += chunk);
99823
99824                 res.on('end', () => resolve(data));
99825
99826             });
99827
99828             req.on('error', reject);
99829
99830             req.setTimeout(2000, () => { req.destroy(); reject(new Error('timeout')); });
99831
99832         });
99833
99834         const vi = JSON.parse(versionJson);
99835
99836         if (vi.webSocketDebuggerUrl) {
99837
99838             _connectTarget(vi.webSocketDebuggerUrl, 'browser');
99839
99840         }
99841
99842     } catch {}
99843     _safeLog('[main] CDP capture started: ' + pageCount + ' page(s) + browser');
99844     _started = true;
99845
99846     // 不在此处 onActive 一 等首条 CDP 数据到达才激活 (防中间空窗导致丢消息)
99847
99848
99849
99850     // 轮询新 target
99851
99852     const _poll = setInterval(async () => {
99853
99854         try {
99855
99856             const resp: string = await new Promise<string>((resolve, reject) => {
```

```
99858         const req = http.get(`http://127.0.0.1:${PORT}/json/list`, (res) => {
99859
99860             let data = '';
99861
99862             res.on('data', (chunk: string) => data += chunk);
99863
99864             res.on('end', () => resolve(data));
99865
99866         });
99867
99868         req.on('error', reject);
99869
99870         req.setTimeout(2000, () => { req.destroy(); });
99871
99872     });
99873
99874     const list: any[] = JSON.parse(resp);
99875
99876     const known = new Set(allSockets.map(s => s.url));
99877
99878     for (const t of list) {
99879
99880         if (t.type === 'page' && t.webSocketDebuggerUrl && !known.has(t.webSocketDebugger
99881
99882             _connectTarget(t.webSocketDebuggerUrl, 'new:' + (t.url || '').slice(0, 60));
99883
99884         }
99885
99886     }
99887
99888     } catch {}
99889
99890 }, 8000);
99891
99892
99893
99894 win.on('closed', () => {
99895
99896     clearInterval(_poll);
99897
99898     for (const s of allSockets) { try { s.close(); } catch {} }
99899
99900     allSockets.length = 0;
99901
99902 });
99903
99904 } catch (err: any) {
99905
99906     _safeLog('[main] CDP capture init failed (retry in 2s):', err.message);
99907     setTimeout(_doCapture, 2000);
```

```
99908
99909     }
99910
99911   };
99912
99913
99914
99915   // 延迟启动: CDP 端口在 DevTools open 之后才完全就绪
99916
99917   setTimeout(_doCapture, 2000);
99918
99919   setTimeout(() => { if (!_started) _doCapture(); }, 5000);
99920
99921 }
99922
99923
99924 // ===== File: shell/wq-ping.ts =====
99925 // Copyright (C) 2025-2026 Sichuan Dream Technology Co., Ltd. All Rights Reserved.
99926
99927 // =====
99928 // wq-ping.ts — 统计上报机（轻量，主进程独立运行）
99929 //
99930 // 职责：
99931 //   1. 生成/持久化 device_id (UUID v4) + factory_version 到 Data/alphal/
99932 //   2. 累计使用时长：跨重启持久化到 Data/alphal/cumulative_seconds
99933 //      → 每次 ping 上报绝对值 → 服务端 max(old, new) 接受
99934 //   3. 定期 POST /api/wq/ping（首次 30~120s 随机抖动，后续按服务端建议）
99935 //   4. 失败指数退避重试
99936 //
99937 // 铁律：
99938 //   - 不阻塞启动，不阻塞渲染进程
99939 //   - 网络错误静默处理（不影响 IDE 功能）
99940 //   - 使用 Node.js https 模块（Electron 主进程可用，零依赖）
99941 // =====
99942
99943 import * as fs from 'fs';
99944 import * as path from 'path';
99945 import * as os from 'os';
99946 import * as crypto from 'crypto';
99947 import * as https from 'https';
99948 import { safeStorage } from 'electron';
99949 import { APP_VERSION } from '../version';
99950 import { getAuthPhone } from '../auth-state';
99951
99952 // —— 常量 ——
99953 const PING_API_HOST = 'direct-cn.gh555.com';
99954 const PING_API_PATH = '/api/wq/ping';
99955 const GOOD_SLG = 'qqide';
99956 const DISTRIBUTION = 'gh555.com'; // 发行渠道（当前唯一，将来可扩展）
99957 const PING_JITTER_MIN_MS = 5_000; // 首次 ping 最小延迟
```

```
99958 const PING_JITTER_MAX_MS = 15_000; // 首次 ping 最大延迟
99959 const PING_FALLBACK_SEC = 43_200; // 兜底间隔 12 小时
99960 const RETRY_MIN_MS = 60_000;
99961 const RETRY_MAX_MS = 3_600_000; // 最大 1 小时
99962
99963 // —— 状态 ——
99964 let _deviceId = '';
99965 let _factoryVersion = '';
99966 let _cumulativeSeconds = 0; // 上次持久化的累计秒数
99967 let _sessionStartedAt = Date.now(); // 本次进程启动时间
99968 let _stopped = false;
99969 let _retryDelayMs = RETRY_MIN_MS;
99970 let _timer: ReturnType<typeof setTimeout> | null = null;
99971 let _userDataPath = ''; // ★ portable.userData, 启动时注入
99972
99973 // —— 磁盘日志（调试用，零依赖，原子 append） ——
99974 let _logPath = '';
99975 function pingLog(msg: string): void {
99976     if (!_logPath) {
99977         const base = _userDataPath || path.join(path.dirname(process.execPath), 'Data');
99978         _logPath = path.join(base, 'alpal', 'wq-ping.log');
99979         try { fs.mkdirSync(path.dirname(_logPath), { recursive: true }); } catch (_) {}
99980     }
99981     const line = new Date().toISOString() + ' ' + msg + '\n';
99982     try { fs.appendFileSync(_logPath, line); } catch (_) {}
99983 }
99984
99985 // —— 持久化路径 ——
99986 function alpalDir(): string {
99987     // ★ 优先用注入的 userData 路径（与 main.ts 一致），
99988     // 兜底用 execPath 旁 Data/alpal（绿色包兼容）
99989     const base = _userDataPath || path.join(path.dirname(process.execPath), 'Data');
99990     const dataDir = path.join(base, 'alpal');
99991     try { fs.mkdirSync(dataDir, { recursive: true }); } catch (_) {}
99992     return dataDir;
99993 }
99994
99995 function deviceIdPath(): string {
99996     return path.join(alpalDir(), 'device_id');
99997 }
99998
99999 function factoryVersionPath(): string {
100000     return path.join(alpalDir(), 'factory_version');
100001 }
100002
100003 function cumulativeSecondsPath(): string {
100004     return path.join(alpalDir(), 'cumulative_seconds');
100005 }
100006
100007 // —— Device ID ——
```

```
100008 function loadOrCreateDeviceId(): string {
100009     const fp = deviceIdPath();
100010     try {
100011         if (fs.existsSync(fp)) {
100012             const raw = fs.readFileSync(fp, 'utf8').trim();
100013             if (/^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/i.test(raw)) {
100014                 return raw.toLowerCase();
100015             }
100016         }
100017     } catch (_) {}
100018
100019     const bytes = crypto.randomBytes(16);
100020     bytes[6] = (bytes[6] & 0x0f) | 0x40;
100021     bytes[8] = (bytes[8] & 0x3f) | 0x80;
100022     const hex = bytes.toString('hex');
100023     const uuid = `${hex.slice(0,8)}-${hex.slice(8,12)}-${hex.slice(12,16)}-${hex.slice(16,20)}-${hex.sli
100024
100025     try { fs.writeFileSync(fp, uuid, 'utf8'); } catch (_) {}
100026     return uuid;
100027 }
100028
100029 // —— 工厂版本 ——
100030 function loadOrCreateFactoryVersion(): string {
100031     const fp = factoryVersionPath();
100032     try {
100033         if (fs.existsSync(fp)) {
100034             const raw = fs.readFileSync(fp, 'utf8').trim();
100035             if (raw.length > 0 && raw.length <= 50) return raw;
100036         }
100037     } catch (_) {}
100038
100039     const fv = APP_VERSION;
100040     try { fs.writeFileSync(fp, fv, 'utf8'); } catch (_) {}
100041     return fv;
100042 }
100043
100044 // —— 累计秒数（跨重启持久化） ——
100045 function loadCumulativeSeconds(): number {
100046     const fp = cumulativeSecondsPath();
100047     try {
100048         if (fs.existsSync(fp)) {
100049             const raw = fs.readFileSync(fp, 'utf8').trim();
100050             const n = parseInt(raw, 10);
100051             if (n >= 0 && n < 315360000) return n; // 0 ~ 10年
100052         }
100053     } catch (_) {}
100054     return 0;
100055 }
100056
100057 function persistCumulativeSeconds(total: number): void {
```



```
100058     try {
100059         fs.writeFileSync(cumulativeSecondsPath(), String(Math.floor(total)), 'utf8');
100060     } catch (_) {}
100061 }
100062
100063 // 当前累计 = 上次持久化值 + 本次进程存活时间
100064 function currentTotalSeconds(): number {
100065     return _cumulativeSeconds + Math.floor((Date.now() - _sessionStartedAt) / 1000);
100066 }
100067
100068 // —— Doer ID (从 auth.enc 读手机号) ——
100069 function authFilePath(): string {
100070     return path.join(alphaDir(), 'auth.enc');
100071 }
100072
100073 function readDoerID(): string {
100074     // ★ 第一优先: 共享内存 (由 main.ts IPC handler 更新, 免 safeStorage)
100075     const cached = getAuthPhone();
100076     if (cached && /\d{7,20}$/.test(cached)) return cached;
100077
100078     // ★ 第二优先: safeStorage 直接解密 (兜底)
100079     try {
100080         if (safeStorage.isEncryptionAvailable()) {
100081             const fp = authFilePath();
100082             if (fs.existsSync(fp)) {
100083                 const encrypted = fs.readFileSync(fp);
100084                 const auth = JSON.parse(safeStorage.decryptString(encrypted));
100085                 if (auth && auth.phone && /\d{7,20}$/.test(auth.phone)) return auth.phone;
100086             }
100087         }
100088     } catch (_) {}
100089
100090     // ★ 第三优先: 纯文本 phone.txt 终极兜底 (防 DPAPI 跨目录/跨用户失效)
100091     try {
100092         const phoneFile = path.join(alphaDir(), 'phone.txt');
100093         if (fs.existsSync(phoneFile)) {
100094             const phone = fs.readFileSync(phoneFile, 'utf8').trim();
100095             if (phone && /\d{7,20}$/.test(phone)) return phone;
100096         }
100097     } catch (_) {}
100098
100099     return '';
100100 }
100101
100102 // —— 收集设备信息 ——
100103 function collectPingBody(): string {
100104     const nowSec = Math.floor(Date.now() / 1000);
100105     const totalSec = currentTotalSeconds();
100106
100107     const body: Record<string, unknown> = {
```

```
100108     good_slg:      GOOD_SLG,
100109     device_id:      _deviceId,
100110     doer_id:         readDoerID(),
100111     total_seconds:   totalSec,
100112     event_time:      nowSec,
100113     client_ver:      APP_VERSION,
100114     factory_version: _factoryVersion,
100115     distribution:    DISTRIBUTION,
100116     pkg_name:        'qqqide',
100117     pkg_display_name: 'qqqide',
100118     pkg_publisher:   'gh555.com',
100119     os_platform:     os.platform(),
100120     os_arch:         os.arch(),
100121     os_ver:          os.release().slice(0, 30),
100122     cpu_cores:       os.cpus().length,
100123     mem_mb:          Math.round(os.totalmem() / (1024 * 1024)),
100124   };
100125
100126   return JSON.stringify(body);
100127 }
100128
100129 // —— 发送 ping ——
100130 function sendPing(): Promise<{ ok: boolean; minNextPingAt?: number }> {
100131   return new Promise((resolve) => {
100132     pingLog('sendPing START host=' + PING_API_HOST);
100133     const bodyStr = collectPingBody();
100134
100135     const req = https.request({
100136       hostname: PING_API_HOST,
100137       port: 443,
100138       path: PING_API_PATH,
100139       method: 'POST',
100140       headers: {
100141         'Content-Type': 'application/json',
100142         'Content-Length': Buffer.byteLength(bodyStr),
100143       },
100144       timeout: 10_000,
100145     }, (res) => {
100146       let data = '';
100147       res.on('data', (chunk: Buffer) => { data += chunk.toString(); });
100148       res.on('end', () => {
100149         try {
100150           const json = JSON.parse(data);
100151           resolve({
100152             ok: json.ok === true,
100153             minNextPingAt: json.min_next_ping_at,
100154           });
100155         } catch {
100156           resolve({ ok: false });
100157         }
100158       });
100159     });
100160   });
100161 }
```

```
100158     });
100159   });
100160
100161   req.on('error', (e: Error) => {
100162     pingLog(' sendPing ERROR: ' + e.message);
100163     resolve({ ok: false, error: e.message } as any);
100164   });
100165   req.on('timeout', () => {
100166     pingLog(' sendPing TIMEOUT');
100167     req.destroy(); resolve({ ok: false, error: 'timeout' } as any);
100168   });
100169
100170   req.write(bodyStr);
100171   req.end();
100172 });
100173 }
100174
100175 // —— 调度 ——
100176 function scheduleNext(delaySec: number) {
100177   if (_stopped) return;
100178   if (_timer) clearTimeout(_timer);
100179   _timer = setTimeout(pingCycle, delaySec * 1000);
100180 }
100181
100182 async function pingCycle() {
100183   if (_stopped) return;
100184
100185   try {
100186     pingLog('sending doer=' + (readDoerID() || 'none') + ' dev=' + _deviceId.slice(0,8));
100187     const res = await sendPing();
100188     pingLog('result ok=' + res.ok + ' err=' + ((res as any).error || ''));
100189     if (res.ok) {
100190       _retryDelayMs = RETRY_MIN_MS;
100191
100192       // ★ 成功后立即持久化当前累计值（原子推进）
100193       persistCumulativeSeconds(currentTotalSeconds());
100194
100195       if (res.minNextPingAt && res.minNextPingAt > 0) {
100196         const nowSec = Math.floor(Date.now() / 1000);
100197         const delaySec = Math.max(60, res.minNextPingAt - nowSec);
100198         scheduleNext(delaySec);
100199       } else {
100200         scheduleNext(PING_FALLBACK_SEC);
100201       }
100202     } else {
100203       // ★ 网络错误/超时/JSON解析失败 → 指数退避重试，不用 12h 兜底
100204       _timer = setTimeout(pingCycle, _retryDelayMs);
100205       _retryDelayMs = Math.min(_retryDelayMs * 2, RETRY_MAX_MS);
100206     }
100207   } catch (_) {
```

```
100208 // 理论上 sendPing 不抛异常（全部 resolve），保留此路径以防未来变更
100209 if (!_stopped) {
100210     _timer = setTimeout(pingCycle, _retryDelayMs);
100211     _retryDelayMs = Math.min(_retryDelayMs * 2, RETRY_MAX_MS);
100212 }
100213 }
100214 }
100215
100216 // —— 公开 API ——
100217
100218 /** 启动统计上报机。调用一次，幂等。
100219 * @param userDataPath portable.userData（与 main.ts AUTH_FILE 同根） */
100220 export function startWqPing(userDataPath?: string): void {
100221     pingLog('startWqPing CALLED userDataPath=' + (userDataPath || 'none') + ' _deviceId=' + (_deviceId |
100222     if (_deviceId) { pingLog('already started, skip'); return; }
100223     if (userDataPath) _userDataPath = userDataPath;
100224
100225     _deviceId = loadOrCreateDeviceId();
100226     _factoryVersion = loadOrCreateFactoryVersion();
100227     _cumulativeSeconds = loadCumulativeSeconds();
100228     _sessionStartedAt = Date.now();
100229     _stopped = false;
100230
100231     pingLog('STARTED dev=' + _deviceId.slice(0, 8) + ' ph=' + (readDoerID() || 'none'));
100232     const jitter = PING_JITTER_MIN_MS + Math.random() * (PING_JITTER_MAX_MS - PING_JITTER_MIN_MS);
100233     _timer = setTimeout(pingCycle, jitter);
100234 }
100235
100236 /** 登录成功后调用：重置退避 + 立即发 ping（带上 doer_id）。 */
100237 export function notifyAuthReady(): void {
100238     pingLog('notifyAuth dev=' + (_deviceId ? _deviceId.slice(0, 8) : 'none') + ' stop=' + _stopped);
100239     if (_stopped || !_deviceId) return;
100240     _retryDelayMs = RETRY_MIN_MS;
100241     if (_timer) clearTimeout(_timer);
100242     _timer = setTimeout(pingCycle, 2_000); // 2s 后发，给 setAuthPhone 留时间
100243 }
100244
100245 /** 停止统计上报机。退出前持久化累计秒数。 */
100246 export function stopWqPing(): void {
100247     _stopped = true;
100248     if (_timer) {
100249         clearTimeout(_timer);
100250         _timer = null;
100251     }
100252     // ★ 退出前写入当前累计（防 crash 丢时间）
100253     if (_deviceId) {
100254         persistCumulativeSeconds(currentTotalSeconds());
100255     }
100256 }
100257
```